

```
In [1]: import pandas as pd
import numpy as np
import sklearn
import imblearn

import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings(action = 'ignore')

from IPython.display import Image

for i in [pd, np, sklearn, imblearn, mpl, sns]:
    print(i.__name__, i.__version__)
```

```
pandas 2.1.4
numpy 1.26.4
sklearn 1.2.2
imblearn 0.11.0
matplotlib 3.8.0
seaborn 0.12.2
```

```
In [3]: Image('148.png')
```

Out[3]: 3-4. 클래스 불균형 해소(Class Imbalance)

- 클래스 불균형 문제는 클래스의 출현 빈도의 차이가 심한 상태를 의미합니다.
- 클래스의 불균형으로 인한 성능 저하를 완화하기 위한 방법들을 알아봅니다.
- f1 점수(f1 score), 매크로 재현율(Macro recall)과 같이 소수의 클래스의 분류 정확도를 올리면 성능 향상을 기대할 수 있습니다. 클래스 불균형 해소를 통해 성능의 개선 효과를 기대할 수 있습니다.

$$\text{f1 점수(f1 score)} = 2 \times \frac{\text{Recall} \times \text{Precision}}{\text{Recall} + \text{Precision}}$$

$$\text{매크로 재현율(Macro recall)} = \frac{1}{c} \sum_{i=1}^c \text{Recall}_i, \text{Recall}_i: i \text{ 클래스를 양성으로 했을 때의 재현율(Recall)}$$

Credit Card Fraud Detection

[신용 카드 사기 거래 탐지](#) 데이터 셋입니다.

Name	Description
Time	첫 거래와의 시간 차, 초(second)단위
V1, V2, ... V28	PCA를 통해 처리된 속성 데이터
amount	거래 금액
Class	1: Fraud, 0: Other

284,807 건의 거래 중에서 492 건 만이 Fraud(사기 거래) 입니다.

굉장히 치우침이 심한 데이터셋입니다.

```
In [2]: df_credit = pd.read_pickle('data/credit.pkl')

df_credit.head()
```

Out[2]:

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22	
0	0	-1.359375	-0.072754	2.537109	1.377930	-0.338379	0.462402	0.239624	0.098694	0.363770	...	-0.018311	0.277832	-0.110
1	0	1.191406	0.266113	0.166504	0.448242	0.060028	-0.082336	-0.078796	0.085083	-0.255371	...	-0.225830	-0.638672	0.101
2	1	-1.358398	-1.339844	1.773438	0.379883	-0.503418	1.800781	0.791504	0.247681	-1.514648	...	0.248047	0.771484	0.909
3	1	-0.966309	-0.185181	1.792969	-0.863281	-0.010307	1.247070	0.237549	0.377441	-1.386719	...	-0.108276	0.005272	-0.190
4	2	-1.158203	0.877930	1.548828	0.403076	-0.407227	0.095947	0.592773	-0.270508	0.817871	...	-0.009430	0.798340	-0.137

5 rows × 31 columns



1. Over sampling

- 빈도가 상대적으로 적은 클래스 데이터를

더 생성하여 불균형을 해소하는 방법입니다.

In [4]: `Image('149.png')`

Out[4]: **Random Over Sampling**

샘플할 클래스와 동일한 클래스인 데이터 중에서 임의로 뽑아내는 샘플링 방법입니다.

imblearn.over_sampling.RandomOverSampler

sampling_strategy:

float일 경우, 이진 분류에만 유효합니다. 리샘플링 후 $\frac{\text{소수 클래스의 샘플링 후의 개수}}{\text{다수 클래스의 개수}}$

dict일 경우, {Class: 빈도수} 의 구조. 리샘플링 후의 각 클래스의 샘플 수

'minority': 가장 빈도가 적은 클래스만 리샘플링합니다.

'not majority': 가장 빈도가 높은 클래스를 제외하고 리샘플링

'auto': 자동 - not majority

[Ex.1]

클래스수가 2개인 클래스 불균형 지닌 데이터셋을 만들고,

sampling_strategy 설정에 따라 추출된 데이터의

클래스의 개수를 확인하여 OverSampler의 동작을 확인합니다.

[0.1, 0.5, 1.0, 'minority']

In [3]: '''
코드는 Python의 scikit-learn 라이브러리에서 제공하는
'make_classification' 함수를 사용하여
가상의 분류 데이터를 생성하는 것입니다.
이 함수는 분류 모델을 실험하거나 시각화를 위해
간단한 가상 데이터를 생성하는 데 사용됩니다.

- ``n_samples``: 생성할 샘플의 총 개수입니다.
여기서는 5000개의 샘플을 생성합니다.
- ``n_features``: 생성할 특성의 개수입니다.
이 예제에서는 2개의 특성을 생성합니다.
- ``n_informative``: 클래스를 구분하는 데 유용한 특성의 개수입니다.
이 예제에서는 2개의 특성이 유용합니다.
- ``n_redundant``, ``n_repeated``: 여분의 또는 중복된 특성의 개수를 나타냅니다. 여기서는 0으로 설정되어 있습니다.
- ``n_classes``: 생성할 클래스의 개수입니다.
이 예제에서는 2개의 클래스를 생성합니다.
- ``n_clusters_per_class``: 각 클래스 당 클러스터의 수입니다.
이 예제에서는 각 클래스당 1개의 클러스터를 생성합니다.
- ``weights``: 각 클래스에 대한 가중치를 지정합니다. 이 예제에서는 클래스 0과 클래스 1에 대해 각각 0.01과 0.99의 비율을 가집니다.
- ``class_sep``: 클래스 간의 평균 거리입니다. 더 높은 값은 클래스가 더 분리되어 있음을 의미합니다. 이 예제에서는 0.8로 설정됩니다.
- ``random_state``: 무작위성을 제어하기 위한 시드 값입니다.
같은 시드 값을 사용하면 항상 동일한 결과가 생성됩니다.

``make_classification`` 함수는 두 개의 반환값을 가집니다.
하나를 생성된 특성 데이터 ``X``이고, 다른 하나는 해당 데이터에 대한 클래스 레이블 ``y``입니다. 이 코드에서는 이러한 값을 ``X``와 ``y`` 변수에 할당합니다.

따라서 이 코드는 2차원의 가상 분류 데이터를 생성하고,
이 데이터는 두 클래스를 나타내며,
클래스 간에는 상당한 분리가 있습니다.
클래스 0은 1에 비해 상대적으로 적은 비율을 차지하고 있습니다.
...

```
from sklearn.datasets import make_classification
```

```
X, y = make_classification(
    n_samples=5000, n_features=2, n_informative=2,
    n_redundant=0, n_repeated=0, n_classes=2,
    n_clusters_per_class=1,
    weights=[0.01, 0.99],
    class_sep=0.8, random_state=123
)
```

In [4]:

```
'''
코드는 `imbalanced-learn` 라이브러리에서 제공하는
`RandomOverSampler`를 사용하여 클래스 불균형 문제를
해결하는 방법을 시연합니다.
클래스 불균형은 각 클래스가 데이터셋에서 불균형하게
표현되는 경우를 의미합니다.
이는 예측 모델이 불균형 클래스에 대해 편향되거나
정확도가 떨어질 수 있는 문제를 야기할 수 있습니다.

여기서 사용된 코드의 주요 부분은 다음과 같습니다:

1. `from imblearn.over_sampling import RandomOverSampler`:
   이 오버 샘플링 기법은 소수 클래스의 샘플을 늘려서
   클래스 간의 균형을 맞추는 방법 중 하나입니다.

2. `ros = RandomOverSampler(sampling_strategy=i)` :
   다양한 오버 샘플링 전략을 시도하기 위해
   `RandomOverSampler` 객체를 생성합니다.
   `sampling_strategy` 매개변수는 다양한 값을 가질 수 있습니다.
   `i`가 숫자인 경우, 그 값을 소수 클래스의 비율로 정의합니다.
   예를 들어, 0.5는 소수 클래스의 샘플 수를 다수 클래스의
   50%로 만듭니다. `'minority'`로 설정하면 소수 클래스를
   다수 클래스에 맞추는 것입니다.

3. `X_ros, y_ros = ros.fit_resample(X, y)` :
   `fit_resample` 메서드를 사용하여 오버 샘플링을 수행합니다.
   이는 주어진 샘플 데이터 `X`와 해당 클래스 레이블 `y`를
   사용하여 오버 샘플링된 새로운 데이터 `X_ros`와
   해당 클래스 레이블 `y_ros`를 생성합니다.

4. `pd.concat(class_cnt, axis=1)` :
   클래스 레이블의 빈도를 보여주기 위해
   데이터프레임을 생성합니다. 이 데이터프레임은
   원본 클래스 분포와 각각의 샘플링 전략을 적용한 후의
   클래스 분포를 보여줍니다.

따라서 이 코드는 원본 데이터의 클래스 불균형을 해소하기 위해
다양한 오버 샘플링 전략을 적용하고,
각 전략에 따른 클래스 분포를 비교합니다.
'''

from imblearn.over_sampling import RandomOverSampler
```

```
class_cnt = [pd.Series(y).value_counts().rename('Original')]

for i in [0.1, 0.5, 1.0, 'minority']:
    ros = RandomOverSampler(sampling_strategy=i)
    X_ros, y_ros = ros.fit_resample(X, y)
    class_cnt.append(
        pd.Series(y_ros).value_counts() \
            .rename('Sampling strategy: {}'.format(i))
    )
pd.concat(class_cnt, axis=1)
```

Out[4]:

	Original	Sampling strategy: 0.1	Sampling strategy: 0.5	Sampling strategy: 1.0	Sampling strategy: minority
1	4919	4919	4919	4919	4919
0	81	491	2459	4919	4919

[Ex.2]

클래스수가 3개인 클래스 불균형 지닌 데이터셋을 만들고,

sampling_strategy 설정에 따라 추출된 데이터의 클래스의

개수를 확인하여 OverSampler의 동작을 확인합니다.

```
[{0: 100, 2: 200}, 'minority', 'not majority']
```

In [5]:

```
'''
코드는 이전과 유사하지만, 생성되는 데이터셋의 클래스 개수가
3개인 것이 다릅니다. 이전에는 클래스가 2개였지만,
이번에는 3개입니다. 여전히 `make_classification` 함수를
사용하여 가상의 분류 데이터를 생성하며,
사용되는 매개변수는 다음과 같습니다:

- `n_samples`: 생성할 샘플의 총 개수로,
  이 예제에서는 5000개의 샘플을 생성합니다.
- `n_features`: 생성할 특성의 개수로,
  이 예제에서는 2개의 특성을 생성합니다.
- `n_informative`: 클래스를 구분하는 데 유용한 특성의 개수로,
```

- 이 예제에서는 2개의 특성이 유용합니다.
- ``n_redundant``, ``n_repeated``: 여분의 또는 중복된 특성의 개수를 나타냅니다. 이 예제에서는 0으로 설정되어 있습니다.
 - ``n_classes``: 생성할 클래스의 개수입니다. 이 예제에서는 3개의 클래스를 생성합니다.
 - ``n_clusters_per_class``: 각 클래스 당 클러스터의 수입니다. 이 예제에서는 각 클래스당 1개의 클러스터를 생성합니다.
 - ``weights``: 각 클래스에 대한 가중치를 지정합니다. 이 예제에서는 클래스 0과 클래스 1에 대해 각각 0.01과 0.99의 비율을 가집니다.
 - ``class_sep``: 클래스 간의 평균 거리입니다. 더 높은 값은 클래스가 더 분리되어 있음을 의미합니다. 이 예제에서는 0.8로 설정됩니다.
 - ``random_state``: 무작위성을 제어하기 위한 시드 값입니다. 같은 시드 값을 사용하면 항상 동일한 결과가 생성됩니다.

따라서 이 코드는 2차원의 가상 분류 데이터를 생성하고, 이 데이터는 세 클래스를 나타내며, 클래스 간에는 상당한 분리가 있습니다. 클래스 0은 클래스 1과 클래스 2에 비해 상대적으로 적은 비율을 차지하고 있습니다.

```
'''
from sklearn.datasets import make_classification

X, y = make_classification(
    n_samples=5000, n_features=2, n_informative=2,
    n_redundant=0, n_repeated=0, n_classes=3,
    n_clusters_per_class=1,
    weights=[0.01, 0.99],
    class_sep=0.8, random_state=123
)'''
```

In [6]:

```
'''
코드는 클래스 불균형 문제를 해결하기 위해
다양한 오버 샘플링 전략을 시연합니다.

1. `class_cnt = list([pd.Series(y).value_counts() \
    .rename('Original')])` :
    원본 클래스의 빈도를 저장하기 위한 리스트를 초기화합니다.
    이는 각 클래스의 샘플 수를 원본으로 저장하는데 사용됩니다.'''
```



```

2. `for i in [{0: 100, 2: 200}, 'minority', 'not majority']:`:
    다양한 오버 샘플링 전략을 시도하기 위한 반복문을 설정합니다.
    여기서 사용되는 오버 샘플링 전략은 세 가지입니다.
    첫 번째는 클래스 0을 100개로, 클래스 2를 200개로
    샘플링하는 딕셔너리입니다.
    두 번째는 `minority`로 설정되어 소수 클래스를
    다수 클래스에 맞추는 것입니다.
    세 번째는 `not majority`로 설정되어 다수 클래스가
    아닌 클래스를 다수 클래스에 맞추는 것입니다.
3. `ros = RandomOverSampler(sampling_strategy=i)`:
    현재 반복되고 있는 오버 샘플링 전략을 기반으로
    `RandomOverSampler` 객체를 생성합니다.
4. `X_ros, y_ros = ros.fit_resample(X, y)`:
    `fit_resample` 메서드를 사용하여 오버 샘플링을 수행합니다.
    이전과 동일하게 주어진 샘플 데이터 `X`와
    해당 클래스 레이블 `y`를 사용하여 오버 샘플링된
    새로운 데이터 `X_ros`와 해당 클래스 레이블 `y_ros`를 생성합니다.
5. `pd.Series(y_ros).value_counts() \
    .rename('Sampling strategy: {}'.format(i))`:
    각 전략에 따른 오버 샘플링 후의 클래스 레이블 빈도를 저장합니다.
    각 전략에 대한 레이블 빈도는 데이터프레임에
    추가될 수 있도록 시리즈로 생성됩니다.
6. `pd.concat(class_cnt, axis=1)`:
    원본 클래스 분포와 각 전략에 따른 오버 샘플링 후의
    클래스 분포를 비교하기 위해 데이터프레임을 생성합니다.
    이 데이터프레임은 클래스 레이블의 빈도를 보여주며,
    열은 원본 데이터와 각 전략에 대한
    오버 샘플링 결과를 보여줍니다.

```

따라서 이 코드는 다양한 오버 샘플링 전략을 적용하여
클래스 불균형을 해소하고, 각 전략에 따른
클래스 분포를 비교합니다.

```

'''

class_cnt = list([pd.Series(y).value_counts() \
                    .rename('Original')])

for i in [{0: 100, 2: 200}, 'minority', 'not majority']:
    ros = RandomOverSampler(sampling_strategy=i)
    X_ros, y_ros = ros.fit_resample(X, y)
    class_cnt.append(

```

```
pd.Series(y_ros).value_counts() \
    .rename('Sampling strategy: {}'.format(i))
)
pd.concat(class_cnt, axis=1)
```

Out[6]:

	Original	Sampling strategy: {0: 100, 2: 200}	Sampling strategy: minority	Sampling strategy: not majority
0	64	100	64	4914
1	4914	4914	4914	4914
2	22	200	4914	4914

이후 클래스 불균형 처리 방법에서 예시를 들기 위한 데이터 처리 작업입니다.

df_fraud 데이터셋의 타겟 변수인 Class의 비율을 동일하게 하여,

80%는 학습데이터 df_train으로, 20%는 평가데이터 df_test로 나눕니다.

Class가 1인 경우가 양성(Positive)이고 0이 음성(Negative) 입니다.

```
In [7]: from sklearn.model_selection import train_test_split
df_train, df_test
        = train_test_split(df_credit, train_size=0.8,
                           random_state=123, stratify=df_credit['Class'])
```

```
In [8]: df_train['Class'].value_counts(normalize=True)
```

```
Out[8]: 0    0.998271
        1    0.001729
        Name: Class, dtype: float64
```

[Ex.3]

학습 데이터 df_train에서 대상 변수(Class)를 제외한

모든 변수를 입력 변수로 설정합니다.

Class가 양성(Positive)인 비율을 아래와 같이 변화를 주어

df_train을 Random Over Sampling을 수행하면서,

성능의 차이를 살펴 봅니다.

Over Sampling한 df_train으로 학습하고, df_test로 성능을 측정합니다.

성능 지표는 f1-score 이고,

학습 모델은 로지스틱 회귀(Logistic Regression)입니다.

```
postive_ratio = [0.005, 0.01, 0.015, 0.02, 0.03, 0.05, 0.08, 0.1, 0.15]
```

```
In [9]: '''
코드는 로지스틱 회귀 모델을 사용하여 클래스 불균형을
다루는 방법을 시연하며, 이를 위해 오버 샘플링을 적용합니다.

1. `f1_train_ros`, `f1_test_ros`, `acc_train`, `acc_test`를
   초기화합니다. 이들은 각각 오버 샘플링 전후의 훈련 및
   테스트 세트에 대한 F1 점수와 정확도를 저장합니다.
2. `X_cols`를 통해 'Class' 열을 제외한 모든 열의
   이름을 가져옵니다. 이 열은 입력 특성으로 사용됩니다.
3. 표준화 처리기(`StandardScaler`)와
   로지스틱 회귀 분류기(`LogisticRegression`)를 결합한
   파이프라인(`Pipeline`)을 생성합니다.
   표준화는 입력 데이터의 각 특성을 평균이 0이고
   표준편차가 1인 분포로 변환합니다.
4. `clf_lr.fit(df_train[X_cols], df_train['Class'])`:
   훈련 데이터를 사용하여 로지스틱 회귀 모델을 훈련합니다.
5. 오버 샘플링을 적용하지 않았을 때의 성능을 기록합니다.
   훈련 세트와 테스트 세트에 대한 F1 점수와 정확도를
   각각 `f1_train_ros`, `f1_test_ros`, `acc_train`,
   `acc_test`에 추가합니다.
6. 주어진 오버 샘플링 비율(`postive_ratio`)에 대해
   반복문을 실행합니다. 각 비율에 대해 RandomOverSampler를
   사용하여 훈련 데이터를 오버 샘플링합니다.
   그런 다음 새로운 훈련 데이터로 로지스틱 회귀 모델을
   다시 훈련하고 훈련 및 테스트 세트에 대한
   F1 점수와 정확도를 계산하고 기록합니다.
'''
```

7. 마지막으로, 오버 샘플링 비율에 따른
F1 점수와 정확도를 기록합니다.

따라서 이 코드는 로지스틱 회귀 모델을 사용하여
클래스 불균형을 다루는 방법을 시연하고,
오버 샘플링 비율에 따른 모델 성능을 평가합니다.
...

...

``fit_resample()`` 함수는 ``imbalanced-learn`` 라이브러리에서
제공하는 오버 샘플링을 수행하는 메서드입니다.
이 함수는 주어진 데이터를 샘플링하여 클래스 불균형을
해소하는 데 사용됩니다. 이 함수의 주요 인자와
기능을 설명하겠습니다:

1. **X:** (배열 혹은 데이터프레임) 특성 행렬입니다.
각 행은 데이터 포인트를 나타내며,
각 열은 해당 데이터 포인트의 특성을 나타냅니다.
2. **y:** (배열 또는 시리즈) 타겟 벡터 또는 레이블입니다.
각 데이터 포인트에 대한 클래스 레이블을 나타냅니다.
3. **sampling_strategy:** (문자열, 딕셔너리, float 혹은 callable)
오버 샘플링 전략을 지정합니다.
여러 형태의 값을 가질 수 있습니다.
 - 문자열: 사전 정의된 오버 샘플링 전략 중 하나를
선택할 수 있습니다. 예를 들어, `'minority'`는
소수 클래스를 다수 클래스에 맞추는 것을 의미합니다.
 - 딕셔너리: 각 클래스에 대한 샘플링 비율을 지정합니다.
키는 클래스 레이블이고, 값은 해당 클래스의
샘플링 비율입니다.
 - float: 소수 클래스의 샘플 수를 다수 클래스의 비율로
지정합니다. 예를 들어, `0.5`는 소수 클래스의
샘플 수를 다수 클래스의 50%로 만듭니다.
 - callable: 사용자 지정 함수를 제공하여
샘플링 전략을 정의할 수 있습니다.
4. **return_indices:** (bool) True로 설정하면 오버 샘플링된
샘플의 인덱스를 반환합니다. 기본값은 False입니다.
5. **random_state:** (int 또는 RandomState 인스턴스) 무작위성을
제어하기 위한 시드 값입니다.
동일한 시드 값을 사용하면
항상 동일한 샘플이 생성됩니다.

```

`fit_resample()` 함수는 주어진 샘플링 전략에 따라
오버 샘플링된 새로운 특성 행렬(`X_resampled`)과
해당 클래스 레이블(`y_resampled`)을 반환합니다.
옵션으로 `return_indices=True`로 설정하면
오버 샘플링된 샘플의 인덱스도 함께 반환됩니다.
'''

from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import f1_score, accuracy_score

f1_train_ros, f1_test_ros = list(), list()
acc_train, acc_test = list(), list()
X_cols = [i for i in df_train.columns if i != 'Class']

# 표준화 처리기와 로지스틱 회귀모델을 결합합니다.
clf_lr = make_pipeline(StandardScaler(),
                        LogisticRegression(solver='lbfgs'))
clf_lr.fit(df_train[X_cols], df_train['Class'])

# 오버샘플링하지 않았을 경우의 성능을 기록합니다.
prd = clf_lr.predict(df_train[X_cols])
f1_train_ros.append(f1_score(df_train['Class'], prd))
acc_train.append(accuracy_score(df_train['Class'], prd))

prd = clf_lr.predict(df_test[X_cols])
f1_test_ros.append(f1_score(df_test['Class'], prd))
acc_test.append(accuracy_score(df_test['Class'], prd))

# 제시한 오버샘플링 비율에 따른 성능을 측정합니다.
postive_ratio \
    = [0.005, 0.01, 0.015, 0.02, 0.03, 0.05, 0.08, 0.1, 0.15]

for i in postive_ratio:
    ros = RandomOverSampler(sampling_strategy=i)
    X_ros, y_ros = ros.fit_resample(df_train[X_cols],
                                    df_train['Class'])

    clf_lr = make_pipeline(StandardScaler(),
                            LogisticRegression(solver='lbfgs'))
    clf_lr.fit(X_ros, y_ros)

```

```

prd = clf_lr.predict(X_ros)
f1_train_ros.append(f1_score(y_ros, prd))
acc_train.append(accuracy_score(y_ros, prd))

prd = clf_lr.predict(df_test[X_cols].values)
f1_test_ros.append(f1_score(df_test['Class'], prd))
acc_test.append(accuracy_score(df_test['Class'], prd))

postive_ratio = [df_train['Class'] \
                  .value_counts(normalize=True) \
                  .loc[1]] + postive_ratio

```

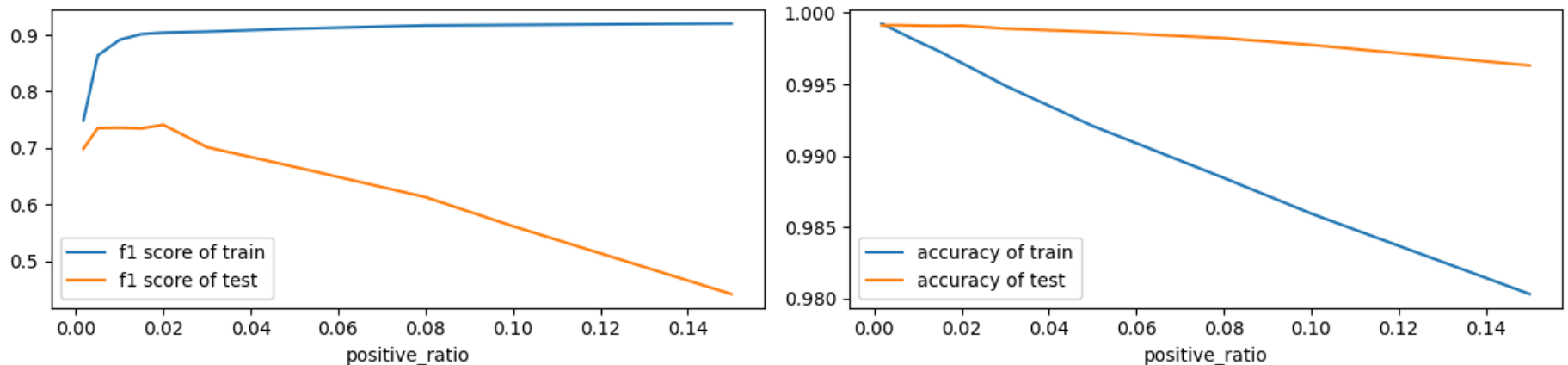
```

In [10]: fig, axes = plt.subplots(1, 2, figsize=(12, 3))
axes[0].plot(postive_ratio, f1_train_ros, label='f1 score of train')
axes[0].plot(postive_ratio, f1_test_ros, label='f1 score of test')
axes[0].set_xlabel('positive_ratio')
axes[0].legend()

axes[1].plot(postive_ratio, acc_train, label='accuracy of train')
axes[1].plot(postive_ratio, acc_test, label='accuracy of test')
axes[1].set_xlabel('positive_ratio')
axes[1].legend()

plt.tight_layout()
plt.show()

```



```
In [5]: Image('150.png')
```

```
Out[5]: SMOTE(Synthetic Minor Over-Sampling Technique)
```

샘플링 클래스에 해당하는 데이터 포인트를 임의로 선택한 후,

선택한 데이터와 동일 클래스인 k 개의 최근접 이웃 데이터 포인트를 구합니다.

k 개의 이웃 중 하나를 골라 두 데이터 포인트간 직선 위의 하나의 포인트를 샘플링합니다.

알고리즘

1. 샘플링 대상 클래스에 해당하는 데이터 포인트를 임의로 선정하고, k 개의 동일 클래스인 최근접 이웃 구합니다.
2. 이웃 중에 무작위로 하나를 선택하고, 두 데이터 포인트를 잇는 직선상의 임의 포인트를 표본으로 만듭니다.
3. 원하는 수만큼의 표본을 취할 때까지 1부터 반복합니다.

imblearn.over_sampling import SMOTE

주요 하이파라미터

이름	설명
<code>k_neighbors</code>	주변 샘플 수

[Ex.3]

SMOTE 알고리즘이 동작하기 위해 2차원의 클래스가

불균형한 2진 데이터를 만듭니다.

SMOTE 후의 표본이 만들어지는 형태를 확인하여

SMOTE의 표본 생성 메카니즘을 시각화해봅니다.

```
In [11]: '''
코드는 다음과 같은 작업을 수행합니다:

1. `make_blobs` 함수를 사용하여 2차원 가상
   데이터를 생성합니다. 이 데이터에는 두 개의
   클러스터가 있으며, 각각의 클러스터는 서로 다른 중심을
   가지고 있습니다. 첫 번째 클러스터는 중심이 [0, 0]이고
   표준 편차가 0.5이며, 두 번째 클러스터는 중심이 [2, 2]이고
   표준 편차가 2입니다. 또한, 첫 번째 클러스터는
   200개의 샘플을, 두 번째 클러스터는 10개의 샘플을 생성합니다.
2. `SMOTE`를 사용하여 클래스 불균형을 해결합니다.
   이때, `sampling_strategy='not majority'`로 설정되어
   소수 클래스를 다수 클래스에 맞추는 오버 샘플링이 수행됩니다.
   `k_neighbors` 매개변수는 SMOTE에서 사용되는
   이웃의 수를 지정합니다. 이 경우에는 3으로 설정되어 있습니다.
3. 원본 데이터와 SMOTE를 적용한 데이터를 시각화합니다.
   이를 위해 두 개의 서브플롯을 생성하고,
   각각의 플롯에 원본 데이터와 SMOTE를 적용한 데이터를
   산점도로 표시합니다. 각 클러스터는 서로 다른 색으로
   표시되며, 클러스터의 중심에는 검은색으로 표시된
   원이 그려집니다.

이러한 시각화를 통해 SMOTE가 어떻게 소수 클래스를
다수 클래스에 가깝게 만드는지를 시각적으로 확인할 수 있습니다.
'''

'''

`make_blobs()` 함수는 scikit-learn 라이브러리에서 제공하는
가상 데이터셋을 생성하는 함수 중 하나입니다.
이 함수를 사용하면 클러스터링이나 분류 모델을 실험하기 위한
간단한 가상 데이터를 만들 수 있습니다.
여러 개의 클러스터가 있는 다차원 데이터셋을 만들어내며,
각 클러스터는 가우시안 분포로부터 생성됩니다.

### 인자:
1. n_samples: 생성할 샘플의 총 개수를 나타냅니다.
2. n_features: 생성할 특성의 개수를 나타냅니다.
   각 샘플은 이러한 특성의 값을 가지게 됩니다.
3. centers: 생성될 클러스터의 수나 중심을 나타내는
   리스트입니다. 예를 들어, centers=[[1,1], [2,2], [3,3]]`는
```


3개의 클러스터를 생성하며,
각각의 중심은 (1,1), (2,2), (3,3)입니다.

4. `cluster_std`: 클러스터 내에서의 표준 편차를 나타내는 리스트입니다. 클러스터마다 서로 다른 표준 편차를 가질 수 있습니다.

리턴값:

``make_blobs()`` 함수는 두 개의 배열을 반환합니다:

1. `X`: 생성된 샘플의 특성 행렬입니다.
각 행은 하나의 샘플을 나타내며,
각 열은 해당 샘플의 특성 값을 나타냅니다.
2. `y`: 각 샘플에 대한 타겟 레이블입니다.
클러스터링을 위한 경우에는 각 샘플이 속한 클러스터를 나타냅니다. 분류를 위한 경우에는 각 샘플이 속한 클래스를 나타냅니다.

``make_blobs()`` 함수는 클러스터링이나 분류 모델을 실험하기 위한 간단한 가상 데이터를 생성하는 데 사용됩니다.

...

...

``SMOTE()`` 함수는 `imbalanced-learn` 라이브러리에서 제공하는 Synthetic Minority Over-sampling Technique(SMOTE)를 구현한 클래스입니다. SMOTE는 클래스 불균형 문제를 해결하기 위한 오버 샘플링 기법 중 하나로, 소수 클래스의 샘플을 합성하여 데이터셋의 클래스 간 균형을 맞춥니다.

여기서 ``SMOTE()`` 클래스의 주요 인자와 메서드를 설명하겠습니다:

인자:

1. `sampling_strategy`: (문자열, 딕셔너리, float 혹은 callable)
오버 샘플링 전략을 지정합니다.
 - 문자열: 사전 정의된 오버 샘플링 전략 중 하나를 선택할 수 있습니다. 예를 들어, ``'minority'``는 소수 클래스를 다수 클래스에 맞추는 것을 의미합니다.
 - 딕셔너리: 각 클래스에 대한 샘플링 비율을 지정합니다. 키는 클래스 레이블이고, 값은 해당 클래스의 샘플링 비율입니다.
 - float: 소수 클래스의 샘플 수를 다수 클래스의 비율로 지정합니다. 예를 들어, `0.5`는 소수 클래스의 샘플 수를 다수 클래스의 50%로 만듭니다.

```

- callable: 사용자 지정 함수를 제공하여
  샘플링 전략을 정의할 수 있습니다.
2. k_neighbors: (int) SMOTE에서 사용되는
  이웃의 수를 지정합니다.
  합성할 인접한 이웃을 찾는 데 사용됩니다.

### 리턴값:
`SMOTE()` 클래스는 `fit_resample()` 메서드를 호출하여
오버 샘플링을 수행합니다. 이 메서드는 샘플링된
새로운 특성 행렬과 해당 클래스 레이블을 반환합니다.

따라서 `SMOTE()` 클래스는 클래스 불균형을 해소하기 위해
사용되며, 주어진 샘플링 전략에 따라 소수 클래스의 샘플을
합성하여 다수 클래스와의 균형을 맞춥니다.
'''

from sklearn.datasets import make_blobs
from imblearn.over_sampling import SMOTE

# 2차원 데이터 샘플 생성
X, y = make_blobs(n_samples=[200, 10],
                  centers=[[0, 0], [2, 2]],
                  cluster_std=[0.5, 2],
                  random_state=123)

# SMOTE 적용
smote = SMOTE(sampling_strategy='not majority',
              k_neighbors=3, random_state=123)

X_resampled, y_resampled = smote.fit_resample(X, y)

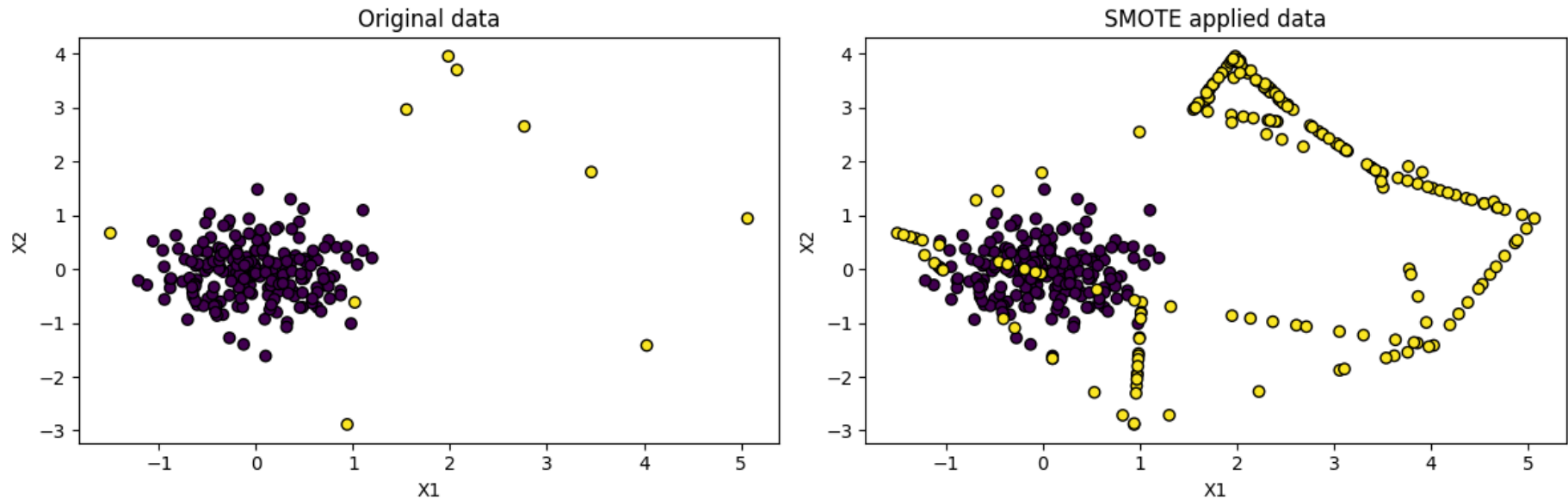
# 원본 데이터 시각화
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

for X_, y_, ax, title in zip([X, X_resampled],
                             [y, y_resampled],
                             axes,
                             ['Original data',
                              'SMOTE applied data']):
    ax.scatter(X_[ :, 0], X_[ :, 1], c=y_, marker='o',
               edgecolors='k')
    ax.set_title(title)

```

```
ax.set_xlabel('X1')
ax.set_ylabel('X2')

plt.tight_layout()
plt.show()
```



[Ex.4]

학습 데이터 df_train에서 대상 변수 Class를 제외한 모든 변수를 입력 변수로 설정합니다.

Class가 양성(Positive)인 비율을 아래와 같이 변화를 주어,

df_train을 SMOTE 알고리즘으로 Over Sampling하고 성능의 차이를 살펴 봅니다.

df_train을 Over Sampling한 데이터로 학습하고, df_test로 성능을 측정합니다.

성능 지표는 f1-score 이고, 학습 모델은 로지스틱 회귀(Logistic Regression)입니다.

```
postive_ratio = [0.005, 0.01, 0.015, 0.02, 0.03, 0.05, 0.08, 0.1, 0.15]
```

```
In [12]: '''
오버샘플링(Over-sampling)은 클래스 불균형 문제를
해결하기 위한 데이터 전처리 기법 중 하나입니다.
클래스 불균형 문제란 데이터셋에서 특정 클래스
(일반적으로 다수 클래스)가 다른 클래스(소수 클래스)보다
훨씬 더 많은 경우를 의미합니다.
이러한 불균형은 머신러닝 모델의 성능에
부정적인 영향을 미칠 수 있습니다.
특히, 모델이 다수 클래스를 더 잘 예측하고,
소수 클래스에 대해서는 성능이 저하되는 현상이
발생할 수 있습니다.

오버샘플링은 소수 클래스의 샘플 수를 증가시켜
클래스 간의 균형을 맞추는 방법입니다.
이를 통해 모델이 소수 클래스도
잘 학습할 수 있도록 합니다.

### 오버샘플링 기법
1. 랜덤 오버샘플링(Random Over-Sampling):
    - 소수 클래스의 샘플을 무작위로 복제하여
      샘플 수를 증가시킵니다.
    - 가장 간단한 오버샘플링 방법이지만,
      과적합(overfitting) 문제를 야기할 수 있습니다.
2. SMOTE (Synthetic Minority Over-sampling Technique):
    - 소수 클래스 샘플의 k-최근접 이웃을 기반으로
      새로운 합성 샘플을 생성합니다.
    - 원본 샘플과 그 이웃 샘플 간의 선형 보간을 통해
      새로운 샘플을 생성합니다.
    - 복제하지 않고 새로운 샘플을 생성하기 때문에
      과적합 문제를 완화할 수 있습니다.
3. ADASYN (Adaptive Synthetic Sampling):
    - SMOTE의 변형 기법으로, 소수 클래스 샘플의
      밀도에 따라 샘플을 적응적으로 생성합니다.
    - 밀도가 낮은 지역에 더 많은 샘플을 생성하여
      데이터 분포를 균일하게 만듭니다.
'''

'''

코드는 로지스틱 회귀 모델을 사용하여 클래스 불균형을
다루는 방법을 시연하고, SMOTE를 적용하여 클래스 불균형을
```

해소하는 방법을 실험합니다.

SMOTE(Synthetic Minority Over-sampling Technique)는 소수 클래스 샘플을 합성하여 데이터셋의 클래스 간 균형을 맞추는 데 사용되는 오버 샘플링 기법입니다. 코드는 SMOTE를 적용하여 클래스 불균형을 해결하고 로지스틱 회귀 모델의 성능을 평가합니다.

1. ``f1_train_smote`, `f1_test_smote`, `acc_train`, `acc_test``를 초기화합니다.
이들은 각각 오버 샘플링 전후의 훈련 및 테스트 세트에 대한 F1 점수와 정확도를 저장합니다.
2. ``X_cols``를 통해 `'Class'` 열을 제외한 모든 열의 이름을 가져옵니다. 이 열은 입력 특성으로 사용됩니다.
3. 표준화 처리기(``StandardScaler``)와 로지스틱 회귀 분류기(``LogisticRegression``)를 결합한 파이프라인(``Pipeline``)을 생성합니다.
표준화는 입력 데이터의 각 특성을 평균이 0이고 표준편차가 1인 분포로 변환합니다.
4. ``clf_lr.fit(df_train[X_cols], df_train['Class'])``:
훈련 데이터를 사용하여 로지스틱 회귀 모델을 훈련합니다.
5. 오버 샘플링을 적용하지 않았을 때의 성능을 기록합니다.
훈련 세트와 테스트 세트에 대한 F1 점수와 정확도를 각각 ``f1_train_smote`, `f1_test_smote`, `acc_train`, `acc_test``에 추가합니다.
6. 주어진 오버 샘플링 비율(``positive_ratio``)에 대해 반복문을 실행합니다. 각 비율에 대해 SMOTE를 사용하여 훈련 데이터를 오버 샘플링합니다.
그런 다음 새로운 훈련 데이터로 로지스틱 회귀 모델을 다시 훈련하고 훈련 및 테스트 세트에 대한 F1 점수와 정확도를 계산하고 기록합니다.
7. 마지막으로, 오버 샘플링 비율에 따른 F1 점수와 정확도를 기록합니다.

코드는 SMOTE를 사용하여 클래스 불균형을 해소하고 모델의 성능을 평가합니다.
...

```
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import f1_score, accuracy_score
```

```

f1_train_smote, f1_test_smote = list(), list()
acc_train, acc_test = list(), list()
X_cols = [i for i in df_train.columns if i != 'Class']

# 표준화 처리기와 로지스틱 회귀모델을 결합합니다.
clf_lr = make_pipeline(StandardScaler(),
                        LogisticRegression(solver='lbfgs'))
clf_lr.fit(df_train[X_cols], df_train['Class'])

# SMOTE처리를 하지 않았을 경우의 성능을 기록합니다.
prd = clf_lr.predict(df_train[X_cols])
f1_train_smote.append(f1_score(df_train['Class'], prd))
acc_train.append(accuracy_score(df_train['Class'], prd))

prd = clf_lr.predict(df_test[X_cols])
f1_test_smote.append(f1_score(df_test['Class'], prd))
acc_test.append(accuracy_score(df_test['Class'], prd))

# 제시한 오버샘플링 비율에 따른 성능을 측정합니다.
postive_ratio \
    = [0.005, 0.01, 0.015, 0.02, 0.03, 0.05, 0.08, 0.1, 0.15]

for i in postive_ratio:
    ros = SMOTE(sampling_strategy=i)
    X_ros, y_ros = ros.fit_resample(df_train[X_cols],
                                    df_train['Class'])
    clf_lr = make_pipeline(StandardScaler(),
                            LogisticRegression(solver='lbfgs'))
    clf_lr.fit(X_ros, y_ros)

    prd = clf_lr.predict(X_ros)
    f1_train_smote.append(f1_score(y_ros, prd))
    acc_train.append(accuracy_score(y_ros, prd))

    prd = clf_lr.predict(df_test[X_cols].values)
    f1_test_smote.append(f1_score(df_test['Class'], prd))
    acc_test.append(accuracy_score(df_test['Class'], prd))

postive_ratio = [df_train['Class']\

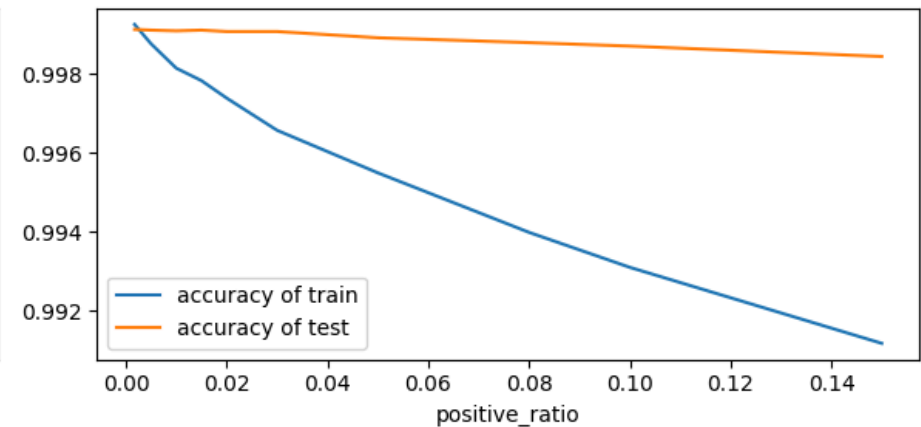
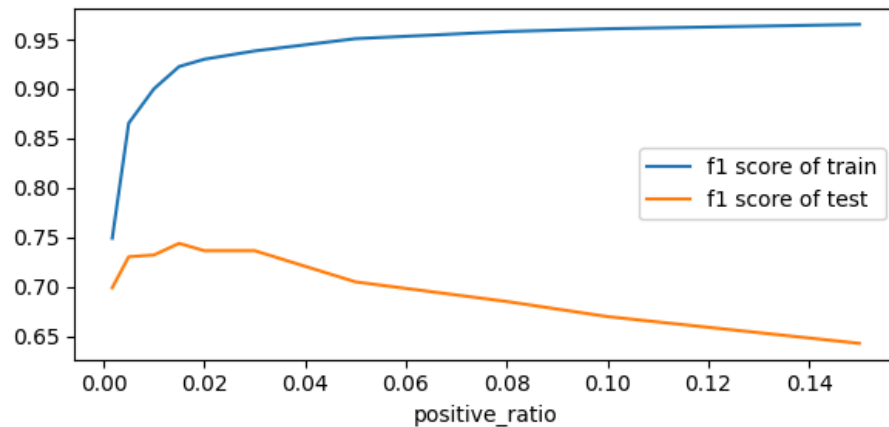
```

```
.value_counts(normalize=True)\
.loc[1]] + positive_ratio
```

```
In [13]: fig, axes = plt.subplots(1, 2, figsize=(12, 3))
axes[0].plot(positive_ratio, f1_train_smote,
             label='f1 score of train')
axes[0].plot(positive_ratio, f1_test_smote,
             label='f1 score of test')
axes[0].set_xlabel('positive_ratio')
axes[0].legend()

axes[1].plot(positive_ratio, acc_train,
             label='accuracy of train')
axes[1].plot(positive_ratio, acc_test,
             label='accuracy of test')
axes[1].set_xlabel('positive_ratio')
axes[1].legend()

plt.tight_layout()
plt.show()
```



ADASYN(Adaptive Synthetic Sampling)

SMOTE 기법을 보완하여 개발된 알고리즘입니다.

샘플을 생성하는 수를 샘플 간 거리에 비례하도록 하여

밀도가 낮은 구역에 샘플이 더 확보되도록 합니다.

```
In [14]: '''
`ADASYN` (Adaptive Synthetic Sampling) 함수는
imbalanced-learn 라이브러리에서 제공하는 클래스입니다.
이 클래스는 소수 클래스의 샘플을 합성하여
클래스 불균형 문제를 해결합니다.

### 주요 인자
1. sampling_strategy: (str, float,
    dict 또는 callable, 기본값='auto')
    - 샘플링 전략을 지정합니다.
      다양한 형태의 값을 가질 수 있습니다.
    - str:
      - ``'minority'``: 소수 클래스 샘플 수를 다수 클래스
        샘플 수와 동일하게 만듭니다.
      - ``'not minority'``: 다수 클래스 샘플 수를 기준으로
        소수 클래스 샘플 수를 증가시킵니다.
      - ``'not majority'``: 다수 클래스 샘플 수를
        변경하지 않고, 나머지 클래스의
        샘플 수를 증가시킵니다.
      - ``'all'``: 모든 클래스의 샘플 수를
        다수 클래스 샘플 수와 동일하게 만듭니다.
    - float: 소수 클래스 샘플 수를 다수 클래스 샘플 수의
      지정된 비율로 설정합니다.
      예를 들어, 0.5는 소수 클래스 샘플 수를
      다수 클래스 샘플 수의 50%로 설정합니다.
    - dict: 각 클래스에 대한 샘플링 비율을 지정합니다.
      키는 클래스 레이블이고, 값은 해당 클래스의
      샘플링 비율입니다.
    - callable: 사용자 지정 함수를 제공하여
      샘플링 전략을 정의할 수 있습니다.
2. n_neighbors: (int 또는 object, 기본값=5)
    - 새로 합성된 샘플을 생성할 때 사용할
      k-최근접 이웃의 수를 지정합니다.
      이웃을 찾기 위해 사용할 k 값입니다.
    - ``n_neighbors``가 int일 경우,
      KNeighborsMixin을 통해 k-최근접 이웃을 찾습니다.
```



```

3. n_jobs: (int, 기본값=None)
    - 학습에 사용할 CPU 코어 수를 지정합니다.
      -1로 설정하면 모든 가용 코어를 사용합니다.

### 메서드

#### fit_resample(X, y)
이 메서드는 주어진 데이터셋을 오버샘플링하여
클래스 불균형 문제를 해결합니다.

- 인자:
    - `X`: (array-like, shape = [n_samples, n_features])
      - 입력 데이터셋입니다. n_samples는 샘플의 수,
        n_features는 특성의 수를 나타냅니다.
    - `y`: (array-like, shape = [n_samples])
      - 입력 데이터셋의 클래스 레이블입니다.

- 리턴값:
    - `X_resampled`: (array-like, shape
                      = [n_samples_new, n_features])
      - 오버샘플링된 입력 데이터셋입니다.
    - `y_resampled`: (array-like, shape = [n_samples_new])
      - 오버샘플링된 입력 데이터셋의 클래스 레이블입니다.
    ...

...

ADASYN(Adaptive Synthetic Sampling) 기법은
클래스 불균형 문제를 해결하기 위해 개발된
오버샘플링 기법 중 하나입니다.
ADASYN은 소수 클래스의 샘플을 합성하여
다수 클래스와의 균형을 맞추는 방법으로,
SMOTE(Synthetic Minority Over-sampling Technique)와
유사하지만, 몇 가지 차별화된 점이 있습니다.

### 주요 개념 및 특징
1. 적응형 샘플링:
    - ADASYN은 소수 클래스 내에서 불균형이 심한
      부분(즉, 분포가 더 드문 부분)을
      중점적으로 샘플링합니다.
    - 이는 새로운 소수 클래스 샘플을 합성할 때,
      소수 클래스 내에서도 더욱 희소한 지역에

```

더 많은 샘플을 추가하여 분포를 균일하게 만드는 데 도움을 줍니다.

2. 데이터의 지역 밀도 반영:

- 각 소수 클래스 샘플의 k -최근접 이웃을 사용하여 밀도를 계산합니다.
- 밀도가 낮은 지역에 속한 소수 클래스 샘플 주위에 더 많은 합성 샘플을 생성합니다.
- 이를 통해 데이터의 지역적 밀도를 반영합니다.

3. 가중치 기반의 샘플링:

- 샘플을 생성할 때 각 소수 클래스 샘플에 가중치를 부여하여, 가중치가 높은 샘플 주위에 더 많은 합성 샘플을 생성합니다.
- 이 가중치는 소수 클래스 샘플의 k -최근접 이웃 중에서 다수 클래스 샘플의 비율에 따라 결정됩니다.

ADASYN의 동작 과정

1. 소수 클래스 샘플의 k -최근접 이웃 찾기:

- 각 소수 클래스 샘플에 대해 k -최근접 이웃을 찾습니다.
- 이웃 중에서 다수 클래스 샘플의 비율을 계산합니다.

2. 가중치 계산:

- 각 소수 클래스 샘플에 대해 가중치를 부여합니다.
- 이 가중치는 다수 클래스 샘플의 비율에 비례합니다.
- 즉, 다수 클래스 샘플이 많을수록 해당 소수 클래스 샘플의 가중치는 높아집니다.

3. 합성 샘플 생성:

- 가중치를 기반으로 합성 샘플을 생성합니다.
- 밀도가 낮은 지역에 더 많은 샘플을 추가하여 분포를 균일하게 만듭니다.
- 새로운 샘플은 기존 소수 클래스 샘플과 그 이웃 샘플의 차이를 랜덤 가중치로 곱하여 생성합니다.

코드 예시

```
from imblearn.over_sampling import ADASYN
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt
```

```
# 2차원 데이터 샘플 생성
```

```
X, y
```

```
    = make_blobs(n_samples=[200, 10],
                 centers=[[0, 0], [2, 2]],
```

```

        cluster_std=[0.5, 2],
        random_state=123)

# ADASYN 적용
adasyn = ADASYN(sampling_strategy='not majority',
                 n_neighbors=3, random_state=123)
X_resampled, y_resampled = adasyn.fit_resample(X, y)

# 원본 데이터 시각화
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

for X_, y_, ax, title in zip([X, X_resampled],
                              [y, y_resampled], axes,
                              ['Original data', 'ADASYN applied data']):
    ax.scatter(X_[ :, 0], X_[ :, 1], c=y_,
               marker='o', edgecolors='k')
    ax.set_title(title)
    ax.set_xlabel('X1')
    ax.set_ylabel('X2')

plt.tight_layout()
plt.show()

```

이 코드는 `make\_blobs`를 사용하여 가상 데이터를 생성하고, ADASYN을 적용하여 소수 클래스 샘플을 합성합니다. 원본 데이터와 ADASYN을 적용한 데이터를 시각화하여 비교합니다.

요약

ADASYN은 소수 클래스의 밀도가 낮은 지역에 중점을 두어 합성 샘플을 생성함으로써 데이터셋의 클래스 불균형 문제를 해결하는 효과적인 방법입니다.

이는 모델이 소수 클래스의 희소한 부분을 더 잘 학습하도록 도와주어 성능을 향상시킬 수 있습니다.

...

이 코드는 ADASYN(Adaptive Synthetic Sampling) 기법을 사용하여 클래스 불균형 문제를 해결하고, 그 결과를 시각화합니다.

1. 데이터 생성

- `make_blobs` 함수는 가상 데이터를 생성합니다.
- `n_samples=[200, 10]`은 첫 번째 클래스에 200개의 샘플, 두 번째 클래스에 10개의 샘플을 생성합니다.
- `centers=[[0, 0], [2, 2]]`는 두 클러스터의 중심을 각각 (0,0)과 (2,2)로 설정합니다.
- `cluster_std=[0.5, 2]`는 각 클러스터의 표준 편차를 지정합니다. 첫 번째 클러스터는 0.5, 두 번째 클러스터는 2로 설정됩니다.
- `random_state=123`는 무작위성을 제어하여 동일한 결과를 재현할 수 있도록 합니다.

2. ADASYN 적용

- `ADASYN` 클래스는 ADASYN 기법을 사용하여 소수 클래스를 오버샘플링합니다.
- `sampling_strategy='not majority'`는 소수 클래스를 다수 클래스의 수에 맞추어 오버샘플링합니다.
- `n_neighbors=3`는 샘플을 합성할 때 고려할 이웃의 수를 설정합니다.
- `fit_resample(X, y)`는 원본 데이터 `X`와 레이블 `y`를 사용하여 오버샘플링을 수행하고, 새로운 데이터와 레이블을 반환합니다.

3. 데이터 시각화

- 두 개의 서브플롯을 생성하여 원본 데이터와 ADASYN을 적용한 데이터를 시각화합니다.
- `fig, axes = plt.subplots(1, 2, figsize=(12, 4))`는 1행 2열의 서브플롯을 생성하고, 전체 크기를 12x4인치로 설정합니다.
- `for` 루프는 원본 데이터(`X, y`)와 ADASYN 적용 후의 데이터(`X_resampled, y_resampled`)를 각각의 서브플롯에 그립니다.
 - `ax.scatter(X[:, 0], X[:, 1], c=y_, marker='o', edgecolors='k')`는 산점도를 그립니다. 각 클래스는 다른 색으로 표시됩니다.
 - `ax.set_title(title)`는 서브플롯의 제목을 설정합니다.
 - `ax.set_xlabel('X1')`와 `ax.set_ylabel('X2')`는 x축과 y축의 레이블을 설정합니다.
- `plt.tight_layout()`는 서브플롯 간의 간격을 자동으로 조정하여 레이아웃이 겹치지 않도록 합니다.

이 코드는 ADASYN을 사용하여 클래스 불균형을 해소하고,
그 결과를 시각적으로 비교할 수 있도록 합니다.
'''

```
from sklearn.datasets import make_blobs
from imblearn.over_sampling import ADASYN

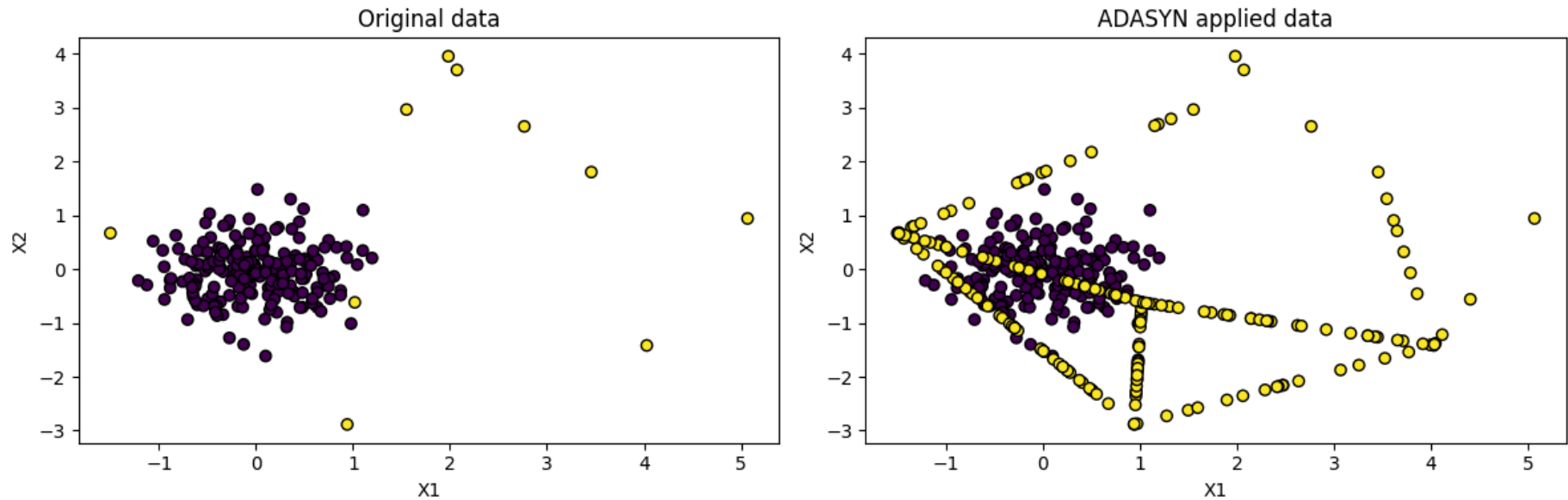
# 2차원 데이터 샘플 생성
X, y = make_blobs(n_samples=[200, 10],
                  centers=[[0, 0], [2, 2]],
                  cluster_std=[0.5, 2],
                  random_state=123)

# SMOTE 적용
smote = ADASYN(sampling_strategy='not majority', \
               n_neighbors=3, random_state=123)
X_resampled, y_resampled = smote.fit_resample(X, y)

# 원본 데이터 시각화
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

for X_, y_, ax, title in zip([X, X_resampled], \
                             [y, y_resampled], axes, \
                             ['Original data', \
                              'ADASYN applied data']):
    ax.scatter(X_[ :, 0], X_[ :, 1], \
               c=y_, marker='o', edgecolors='k')
    ax.set_title(title)
    ax.set_xlabel('X1')
    ax.set_ylabel('X2')

plt.tight_layout()
plt.show()
```

**[Ex.5]**

학습 데이터 df_train에서 대상 변수 Class를 제외한 모든 변수를 입력 변수로 설정합니다.

Class가 양성(Positive)인 비율을 아래와 같이 변화를 주어 df_train을 ADASYN 알고리즘으로

Over Sampling하고 성능의 차이를 살펴 봅니다.

Over Sampling한 데이터로 학습을하고, df_test로 성능을 측정합니다.

성능 지표는 f1-score 이고, 학습 모델은 로지스틱 회귀(Logistic Regression)입니다.

```
postive_ratio = [0.005, 0.01, 0.015, 0.02, 0.03, 0.05, 0.08, 0.1, 0.15]
```

In [15]:

```
'''
이 코드는 로그 회귀 모델을 사용하여 데이터셋을 학습하고
평가하는 작업을 수행하며, ADASYN 기법을 사용하여
데이터 불균형 문제를 해결하는 과정을 포함합니다.
이 과정은 여러 가지의 양성 클래스 비율에 대해
반복됩니다.
'''
```

```

### 초기화 및 데이터 전처리
- `f1_train_adasyn`, `f1_test_adasyn`:
    훈련 및 테스트 데이터셋에 대한 F1 점수를 저장하는 리스트.
- `acc_train`, `acc_test`:
    훈련 및 테스트 데이터셋에 대한 정확도를 저장하는 리스트.
- `X_cols`:
    타겟 열 `Class`를 제외한 모든 특징 열의 리스트를 정의.

### 기본 모델 학습 및 평가
- 표준화(`StandardScaler()`) 및 로그 회귀(`LogisticRegression`)
    모델을 포함하는 파이프라인을 정의합니다.
- 훈련 데이터로 모델을 학습시킵니다.
- 훈련 데이터와 테스트 데이터에 대해 예측을 수행하고,
    F1 점수와 정확도를 리스트에 추가합니다.

### ADASYN을 사용한 데이터 오버샘플링 및 모델 평가
- 다양한 양성 클래스 비율(`positive_ratio`)에 대해 반복합니다.
- ADASYN을 사용하여 지정된 비율로 훈련 데이터를 오버샘플링합니다.
- 새로 생성된 데이터셋으로 파이프라인을 다시 학습시킵니다.
- 오버샘플링된 훈련 데이터와 원래 테스트 데이터에 대해
    예측을 수행하고, F1 점수와 정확도를 리스트에 추가합니다.

### 양성 클래스 비율 업데이트
- 원래 훈련 데이터에서 양성 클래스의 비율을 계산하여
    `positive_ratio` 리스트의 첫 번째 요소로 추가합니다.

이 코드는 모델의 성능을 다양한 양성 클래스 비율에 대해
평가함으로써 데이터 불균형 문제를 해결하려는 시도입니다.
`f1_train_adasyn` 및 `f1_test_adasyn` 리스트는
각각 훈련 및 테스트 데이터셋에서 모델의 F1 점수를 저장하고,
`acc_train` 및 `acc_test` 리스트는 모델의 정확도를 저장합니다.
...

from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import f1_score, accuracy_score

f1_train_adasyn, f1_test_adasyn = list(), list()
acc_train, acc_test = list(), list()

```

```

X_cols = [i for i in df_train.columns if i != 'Class']
clf_lr = make_pipeline(StandardScaler(),
                        LogisticRegression(solver='lbfgs'))
clf_lr.fit(df_train[X_cols], df_train['Class'])

prd = clf_lr.predict(df_train[X_cols])
f1_train_adasyn.append(f1_score(df_train['Class'], prd))
acc_train.append(accuracy_score(df_train['Class'], prd))

prd = clf_lr.predict(df_test[X_cols])
f1_test_adasyn.append(f1_score(df_test['Class'], prd))
acc_test.append(accuracy_score(df_test['Class'], prd))

postive_ratio \
    = [0.005, 0.01, 0.015, 0.02, 0.03, 0.05, 0.08, 0.1, 0.15]
for i in postive_ratio:
    ros = ADASYN(sampling_strategy=i)
    X_ros, y_ros = ros.fit_resample(df_train[X_cols],
                                    df_train['Class'])

    clf_lr = make_pipeline(StandardScaler(),
                            LogisticRegression(solver='lbfgs'))
    clf_lr.fit(X_ros, y_ros)

    prd = clf_lr.predict(X_ros)
    f1_train_adasyn.append(f1_score(y_ros, prd))
    acc_train.append(accuracy_score(y_ros, prd))

    prd = clf_lr.predict(df_test[X_cols].values)
    f1_test_adasyn.append(f1_score(df_test['Class'], prd))
    acc_test.append(accuracy_score(df_test['Class'], prd))

postive_ratio = [df_train['Class']\
                 .value_counts(normalize=True)\
                 .loc[1]] + postive_ratio

```

```

In [16]: fig, axes = plt.subplots(1, 2, figsize=(12, 3))
axes[0].plot(postive_ratio, f1_train_adasyn, label='f1 score of train')
axes[0].plot(postive_ratio, f1_test_adasyn, label='f1 score of test')
axes[0].set_xlabel('positive_ratio')
axes[0].legend()

```

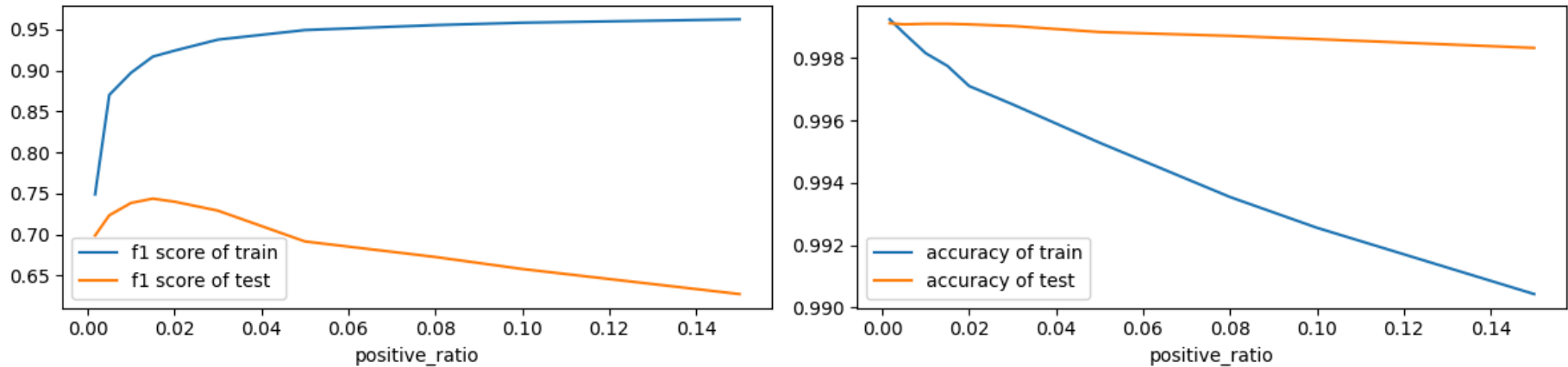


```

axes[1].plot(positive_ratio, acc_train, label='accuracy of train')
axes[1].plot(positive_ratio, acc_test, label='accuracy of test')
axes[1].set_xlabel('positive_ratio')
axes[1].legend()

plt.tight_layout()
plt.show()

```



오버 샘플링의 장단점

장점

- 정보 보존: 소수 클래스의 샘플을 합성하거나 복제하여

데이터의 소수 클래스의 데이터의 특성이 드러나게 하여 정보의 보존 효과를 줍니다.

단점

- 소수 클래스의 샘플을 과도하게 증가시키면 과적합을 유발시킬 수 있습니다.
- 샘플이 증가함에 따라 학습시 계산 비용이 증가합니다.

2. Under sampling

- 다수 클래스(Major class)의 데이터를 줄여 불균형을 해소하는 방법입니다.

In [2]: `Image('151.png')`

Out[2]: **Random Under Sampling**

샘플링 클래스의 데이터를 임의로 선정하여 대상에서 제외시키는 샘플링 방법입니다.

imblearn.under_sampling.RandomUnderSampling

sampling_strategy:

float일 경우, 이진 분류에만 유효합니다. 리샘플링 후 $\frac{\text{소수 클래스의 개수}}{\text{다수 클래스의 샘플링 후의 개수}}$

dict일 경우, {Class: 빈도수}의 구조. 리샘플링 후의 각 클래스의 샘플 수

'majority': 가장 빈도가 높은 클래스만 리샘플링합니다.

'not minority': 가장 빈도가 적은 클래스를 제외하고 리샘플링

'auto': 자동 - not majority

```
In [ ]: '''
아래 코드 실행 결과
원본 데이터 성능:
    F1 (train) = 0.4949832775919732
    Accuracy (train) = 0.96225
원본 데이터 성능:
    F1 (test) = 0.6470588235294118
    Accuracy (test) = 0.964
언더샘플링 성능:'''
```

```
F1 (train) = 0.7684478371501272
Accuracy (train) = 0.8534621578099839
언더샘플링 성능:
F1 (test) = 0.4927536231884058
Accuracy (test) = 0.895
```

결과를 해석하기 위해서는 각 성능 지표(F1 스코어와 정확도)가 무엇을 의미하는지 이해해야 합니다.
또한, 언더샘플링이 모델의 성능에 미치는 영향을 비교해야 합니다.

성능 지표 설명

- 정확도 (Accuracy): 전체 샘플 중에서 올바르게 예측된 샘플의 비율입니다. 다수 클래스가 많은 경우, 정확도는 높게 나타날 수 있지만 소수 클래스에 대한 성능은 잘 반영하지 못할 수 있습니다.
- F1 스코어: 정밀도(Precision)와 재현율(Recall)의 조화 평균입니다. 클래스 불균형 문제에 민감하며, 소수 클래스의 예측 성능을 더 잘 반영합니다.

결과 해석

원본 데이터 성능

- 훈련 데이터:
 - F1 스코어: 0.495
 - 정확도: 0.962
- 테스트 데이터:
 - F1 스코어: 0.647
 - 정확도: 0.964

원본 데이터에서 모델은 높은 정확도를 보이지만, F1 스코어는 상대적으로 낮습니다.
이는 모델이 다수 클래스에 치우친 예측을 하고 있음을 나타냅니다. 높은 정확도는 대부분의 다수 클래스 샘플을 잘 예측하고 있지만, F1 스코어가 낮다는 것은 소수 클래스에 대해 좋은 예측 성능을 보이지 못한다는 의미입니다.

언더샘플링 성능

- 훈련 데이터:
 - F1 스코어: 0.768
 - 정확도: 0.853

- 테스트 데이터 :
 - F1 스코어 : 0.493
 - 정확도 : 0.895

언더샘플링을 적용한 후, 훈련 데이터에서는 F1 스코어가 크게 향상되었지만, 정확도는 감소했습니다.
이는 소수 클래스에 대한 예측 성능이 향상되었음을 의미합니다.
그러나 테스트 데이터에서의 F1 스코어는 원본 데이터보다 낮아졌고, 정확도도 감소했습니다.
이는 언더샘플링이 테스트 데이터에 대해 오버피팅(overfitting)을 일으켰을 가능성을 나타냅니다.

종합적 해석

- 원본 데이터: 높은 정확도를 보이지만, 소수 클래스에 대한 예측 성능이 낮습니다.
- 언더샘플링: 소수 클래스에 대한 예측 성능은 훈련 데이터에서 향상되었지만, 테스트 데이터에서는 성능이 저하되었습니다.
이는 훈련 데이터에서 모델이 소수 클래스를 더 잘 예측하도록 학습되었지만, 테스트 데이터에서는 이러한 성능이 유지되지 않음을 의미합니다.

결론

언더샘플링은 소수 클래스의 예측 성능을 개선할 수 있지만, 테스트 데이터에서 오버피팅 문제가 발생할 수 있습니다.
이를 해결하기 위해서는 다음과 같은 추가적인 방법들을 고려할 수 있습니다:

1. 더 다양한 샘플링 기법:
SMOTE, ADASYN과 같은 오버샘플링 기법이나 혼합 샘플링 기법을 사용해 볼 수 있습니다.
2. 모델 튜닝:
모델의 하이퍼파라미터를 튜닝하여 성능을 개선할 수 있습니다.
3. 앙상블 방법:
여러 모델을 결합하여 예측 성능을 향상시킬 수 있습니다.
4. 교차 검증:
모델의 일반화 성능을 높이기 위해 교차 검증을 사용할 수 있습니다.

결국, 데이터의 특성과 문제의 목적에 맞는
최적의 방법을 선택하는 것이 중요합니다.
'''

```
import numpy as np
import pandas as pd
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import f1_score, accuracy_score
from imblearn.under_sampling import RandomUnderSampler

# 데이터 생성
X, y = make_classification(
    n_samples=5000, n_features=2, n_informative=2,
    n_redundant=0, n_repeated=0, n_classes=2,
    n_clusters_per_class=1,
    weights=[0.95, 0.05],
    class_sep=0.8, random_state=123
)

# 데이터프레임 생성
df = pd.DataFrame(X, columns=['feature1', 'feature2'])
df['Class'] = y

# 데이터 분할
df_train, df_test = train_test_split(df, test_size=0.2, random_state=123)
X_cols = [i for i in df_train.columns if i != 'Class']

# 모델 훈련 및 평가
clf_lr = make_pipeline(StandardScaler(), LogisticRegression(solver='lbfgs'))
clf_lr.fit(df_train[X_cols], df_train['Class'])

# 원본 데이터 성능 평가
prd = clf_lr.predict(df_train[X_cols])
f1_train_orig = f1_score(df_train['Class'], prd)
acc_train_orig = accuracy_score(df_train['Class'], prd)
prd = clf_lr.predict(df_test[X_cols])
f1_test_orig = f1_score(df_test['Class'], prd)
```

```

acc_test_orig = accuracy_score(df_test['Class'], prd)

# 언더샘플링
rus = RandomUnderSampler(sampling_strategy=0.5, random_state=123)
X_rus, y_rus = rus.fit_resample(df_train[X_cols], df_train['Class'])

# 샘플링 후 모델 훈련 및 평가
clf_lr.fit(X_rus, y_rus)
prd = clf_lr.predict(X_rus)
f1_train_rus = f1_score(y_rus, prd)
acc_train_rus = accuracy_score(y_rus, prd)
prd = clf_lr.predict(df_test[X_cols])
f1_test_rus = f1_score(df_test['Class'], prd)
acc_test_rus = accuracy_score(df_test['Class'], prd)

# 결과 출력
print("원본 데이터 성능: F1 (train) =", f1_train_orig, "Accuracy (train) =", acc_train_orig)
print("원본 데이터 성능: F1 (test) =", f1_test_orig, "Accuracy (test) =", acc_test_orig)
print("언더샘플링 성능: F1 (train) =", f1_train_rus, "Accuracy (train) =", acc_train_rus)
print("언더샘플링 성능: F1 (test) =", f1_test_rus, "Accuracy (test) =", acc_test_rus)

```

[Ex.6]

클래스수가 2개인 클래스 불균형 지닌 임의의 데이터셋을 만들고,

sampling_strategy 설정에 따라 추출된 데이터의 클래스의

개수를 확인하여 UnderSampler의 동작을 확인합니다.

```
[0.1, 0.5, 1.0, 'majority']
```

In [17]:

```
'''
이 코드는 `make_classification` 함수를 사용하여
불균형한 이진 분류용 데이터셋을 생성합니다.
`make_classification` 함수는 다양한 파라미터를 통해
데이터를 구성하는 방법을 제어할 수 있습니다.

- `n_samples=5000`: 총 5000개의 샘플(데이터 포인트)을 생성합니다.
- `n_features=2`: 각 샘플에 대해 2개의 특징(피처)을 생성합니다.
'''
```

- ``n_informative=2``: 생성된 2개의 특징 중 2개가 정보성을 갖습니다.
즉, 실제로 클래스 구분에 도움이 되는 특징입니다.
- ``n_redundant=0``: 정보성이 없는 추가적인 중복 특징을 생성하지 않습니다.
- ``n_repeated=0``: 반복된 특징을 생성하지 않습니다.
- ``n_classes=2``: 2개의 클래스로 구성된 이진 분류 문제를 생성합니다.
- ``n_clusters_per_class=1``:
각 클래스는 하나의 클러스터(군집)로 구성됩니다.
이는 각 클래스의 데이터 포인트가 하나의 군집에 집중되도록 만듭니다.
- ``weights=[0.01, 0.99]``: 클래스 비율을 지정합니다.
클래스 0은 전체 샘플의 1%를 차지하고,
클래스 1은 99%를 차지합니다.
이는 매우 불균형한 데이터셋을 생성합니다.
- ``class_sep=0.8``: 클래스 간의 분리도를 설정합니다.
이 값이 클수록 클래스 간의 구분이 뚜렷해집니다.
여기서는 0.8로 설정하여 약간의 분리를 줍니다.

결과

- ``X``: 생성된 데이터 포인트의 특징을 담은 배열.
크기는 (5000, 2)로, 5000개의 샘플과
각 샘플에 대해 2개의 특징을 갖습니다.
- ``y``: 각 샘플의 클래스 레이블을 담은 배열.
크기는 (5000,)로, 각 샘플이 속한 클래스
(0 또는 1)를 나타냅니다.

이 데이터셋은 매우 불균형하게 구성되어 있으며,
이는 불균형한 클래스 분포를 다루는 모델을
평가하는 데 유용할 수 있습니다.
모델이 소수 클래스(여기서는 클래스 0)를
잘 예측할 수 있는지 평가하는데 도움이 됩니다.
...

```
from sklearn.datasets import make_classification
X, y = make_classification(
    n_samples=5000, n_features=2, n_informative=2,
    n_redundant=0, n_repeated=0, n_classes=2,
    n_clusters_per_class=1,
    weights=[0.01, 0.99],
```

```
class_sep=0.8, random_state=123
)
```

In [18]:

```
'''
코드는 `RandomUnderSampler`를 사용하여
불균형한 데이터셋을 다양한 샘플링 전략으로
언더샘플링하고, 각 샘플링 전략에 따른
클래스 분포를 비교하는 작업을 수행합니다.
여기에서 `pd.Series`와 `pd.concat`을 사용하여
결과를 표 형태로 정리합니다.

- `RandomUnderSampler` :
    불균형한 데이터셋의 다수 클래스에서
    무작위로 샘플을 제거하여 클래스 균형을 맞추는 방법.

### 클래스 분포 저장을 위한 리스트 초기화
- `class_cnt` :
    각 샘플링 전략에 따른 클래스 분포를 저장할 리스트.
- `pd.Series(y).value_counts()` :
    원본 데이터셋 `y`의 클래스 분포를 계산.
- `.rename('Original')` :
    이 클래스 분포를 'Original'로 이름을 변경하고,
    리스트의 첫 번째 요소로 추가.

### 다양한 샘플링 전략을 사용한 언더샘플링
- `[0.1, 0.5, 1.0, 'majority']` : 다양한 샘플링 전략 리스트.
    - `0.1`, `0.5`, `1.0` : 각각의 클래스 비율을 지정합니다.
        - `0.1` : 소수 클래스의 비율을
            10%로 맞추는 샘플링 전략.
        - `0.5` : 소수 클래스의 비율을
            50%로 맞추는 샘플링 전략.
        - `1.0` : 소수 클래스의 비율을
            100%로 맞추는 샘플링 전략
            (즉, 클래스 비율을 1:1로 맞춤).
    - `'majority'` :
        다수 클래스의 샘플 수에 맞춰
        소수 클래스의 샘플을 모두 유지하는 샘플링 전략.
- `RandomUnderSampler(sampling_strategy=i)` :
    지정된 전략으로 언더샘플링을 수행할
    `RandomUnderSampler` 객체 생성.
'''
```



```

- `X_rus, y_rus = rus.fit_resample(X, y)` :
  `fit_resample`을 통해 데이터셋 `X`와 `y`를 샘플링.
- `class_cnt.append(pd.Series(y_rus).value_counts()
  .rename('Sampling strategy: {}'.format(i)))` :
  샘플링 후의 클래스 분포를 계산하고,
  샘플링 전략 이름으로 리네임한 후
  `class_cnt` 리스트에 추가.

### 클래스 분포를 표로 결합 및 출력
- `pd.concat(class_cnt, axis=1)` :
  각 샘플링 전략에 따른 클래스 분포를
  세로 방향(컬럼)으로 결합하여
  하나의 데이터프레임으로 만들.
- 최종 결과는 각 샘플링 전략에 따른
  클래스 분포를 비교할 수 있는 표 형태로 출력됩니다.

### 요약
이 코드는 원본 데이터셋과 여러 샘플링 전략
(0.1, 0.5, 1.0, 'majority')을 사용하여
언더샘플링된 데이터셋의 클래스 분포를 비교합니다.
결과는 원본 데이터셋과 각 샘플링 전략에 따른
클래스 분포를 한눈에 볼 수 있는 표로 나타냅니다.
'''

from imblearn.under_sampling import RandomUnderSampler

class_cnt = [pd.Series(y).value_counts().rename('Original')]
for i in [0.1, 0.5, 1.0, 'majority']:
    rus = RandomUnderSampler(sampling_strategy=i)
    X_rus, y_rus = rus.fit_resample(X, y)
    class_cnt.append(
        pd.Series(y_rus).value_counts()\
        .rename('Sampling strategy: {}'.format(i))
    )
pd.concat(class_cnt, axis=1)

```

Out[18]:

	Original	Sampling strategy: 0.1	Sampling strategy: 0.5	Sampling strategy: 1.0	Sampling strategy: majority
1	4919	810	162	81	81
0	81	81	81	81	81

[Ex.7]

클래스수가 3개인 클래스 불균형 지닌 임의의 데이터셋을 만들고,

sampling_strategy 설정에 따라 추출된 데이터의 클래스의

개수를 확인하여 UnderSampler의 동작을 확인합니다.

```
[{0: 100, 1: 1000}, 'majority', 'not minority']
```

In [19]:

```
'''
코드는 `make_classification` 함수를 사용하여
3개의 클래스를 가지는 불균형한 분류 문제를 위한
데이터셋을 생성합니다. 각 파라미터는 데이터의
생성 방식을 제어하며, 그 의미는 다음과 같습니다:

### 파라미터 설명
- `n_samples=5000`:
    총 5000개의 샘플(데이터 포인트)을 생성합니다.
- `n_features=2`:
    각 샘플에 대해 2개의 특징(피처)을 생성합니다.
- `n_informative=2`:
    생성된 2개의 특징 중 2개가 정보성을 가집니다.
    즉, 실제로 클래스 구분에 도움이 되는 특징입니다.
- `n_redundant=0`:
    정보성이 없는 추가적인 중복 특징을 생성하지 않습니다.
- `n_repeated=0`:
    반복된 특징을 생성하지 않습니다.
- `n_classes=3`:
    3개의 클래스로 구성된 분류 문제를 생성합니다.
- `n_clusters_per_class=1`:
    각 클래스는 하나의 클러스터(군집)로 구성됩니다.
'''
```

이는 각 클래스의 데이터 포인트가 하나의 군집에 집중되도록 만듭니다.

- ``weights=[0.05, 0.95]``: 클래스 비율을 지정합니다.
이 경우, 클래스 0과 1의 비율만 지정되어 있습니다.
이는 클래스 0이 전체 샘플의 5%를 차지하고,
클래스 1이 95%를 차지하도록 지정합니다.
명시되지 않은 클래스 2의 비율은 자동으로 조정됩니다.
- ``class_sep=0.8``: 클래스 간의 분리도를 설정합니다.
이 값이 클수록 클래스 간의 구분이 뚜렷해집니다.
여기서는 0.8로 설정하여 약간의 분리를 줍니다.

결과

- ``X``: 생성된 데이터 포인트의 특징을 담은 배열.
크기는 (5000, 2)로, 5000개의 샘플과
각 샘플에 대해 2개의 특징을 갖습니다.
- ``y``: 각 샘플의 클래스 레이블을 담은 배열.
크기는 (5000,)로, 각 샘플이 속한 클래스
(0, 1 또는 2)를 나타냅니다.

요약

이 데이터셋은 3개의 클래스를 가지며,
클래스 0은 전체 샘플의 5%, 클래스 1은 95%를 차지하고,
클래스 2는 나머지 비율을 차지하는 불균형한 분포를 가집니다.
이를 통해 다중 클래스의 불균형한 데이터셋에서
모델의 성능을 평가할 수 있습니다.
다시 한 번 강조하자면, 여기서 ``weights=[0.05, 0.95]``로
설정된 것은 클래스 0과 1의 비율을 의미하며,
실제 데이터 생성 결과에 따라 클래스 2의 비율은
자동으로 계산됩니다.
실제로 3개의 클래스 비율을 정확히 지정하고자 할 경우,
3개의 비율을 모두 명시적으로 설정해야 합니다.
...

```
from sklearn.datasets import make_classification
```

```
X, y = make_classification(
    n_samples=5000, n_features=2, n_informative=2,
    n_redundant=0, n_repeated=0, n_classes=3,
    n_clusters_per_class=1,
    weights=[0.05, 0.95],
```

```
class_sep=0.8, random_state=123
)
```

```
In [20]: '''
코드는 데이터셋 `y`의 클래스 분포를 계산하고,
이를 리스트 `class_cnt`에 저장하는 역할을 합니다.

### 전체 설명
- `pd.Series(y).value_counts()` :
  원본 데이터셋 `y`의 클래스 분포를 계산합니다.
- `.rename('Original')` :
  이 클래스 분포를 'Original'로 이름을 변경합니다.
- `list(...)` : 이 시리즈 객체를 리스트로 변환하여
  `class_cnt` 리스트의 첫 번째 요소로 추가합니다.

### 목적
이 코드는 원본 데이터셋의 클래스 분포를 저장한 후,
이후 단계에서 다양한 샘플링 전략에 따른 클래스 분포를
추가하여 비교할 수 있도록 준비합니다.
이를 통해 데이터셋의 초기 클래스 분포와
샘플링 후의 클래스 분포를 쉽게 비교할 수 있습니다.
'''

class_cnt = list([pd.Series(y).value_counts()\
                  .rename('Original')])
class_cnt
```

```
Out[20]: [1    4715
          0     263
          2      22
          Name: Original, dtype: int64]
```

```
In [21]: '''
코드는 `RandomUnderSampler`를 사용하여
다양한 샘플링 전략으로 데이터셋을 언더샘플링하고,
각 샘플링 전략에 따른 클래스 분포를
비교하는 작업을 수행합니다.
이를 통해 원본 데이터셋과 샘플링된 데이터셋의
클래스 분포를 비교할 수 있습니다.
```

```

### 데이터 준비 및 초기화
# class_cnt 초기화: 원본 데이터셋의 클래스 분포 계산 및 저장

### 언더샘플링 및 클래스 분포 계산
- `for` 루프는 세 가지 샘플링 전략(`{0: 100, 1: 1000}`,
  `'majority'`, `'not minority'`)에 대해 반복합니다.
- 각 반복에서:
  1. `RandomUnderSampler(sampling_strategy=i)` :
    지정된 샘플링 전략으로 언더샘플링을 수행할
    `RandomUnderSampler` 객체를 생성합니다.
  2. `X_rus, y_rus = rus.fit_resample(X, y)` :
    `fit_resample` 메서드를 사용하여
    데이터셋 `X`와 `y`를 샘플링합니다.
    이는 지정된 샘플링 전략에 따라
    클래스 불균형을 조정합니다.
  3. `pd.Series(y_rus).value_counts()` :
    샘플링된 데이터셋 `y_rus`의
    클래스 분포를 계산합니다.
  4. `.rename('Sampling strategy: {}'.format(i))` :
    계산된 클래스 분포를 해당 샘플링 전략 이름으로
    리네임합니다.
  5. `class_cnt.append(...)` :
    이 클래스 분포를 `class_cnt` 리스트에 추가합니다.

### 클래스 분포 결함 및 출력
- `pd.concat(class_cnt, axis=1)` :
  `class_cnt` 리스트에 저장된 모든 클래스 분포 시리즈를
  세로 방향(컬럼)으로 결합하여
  하나의 데이터프레임으로 만듭니다.
- 최종 결과는 각 샘플링 전략에 따른
  클래스 분포를 비교할 수 있는 표 형태로 출력됩니다.

### 각 샘플링 전략 설명
- `{0: 100, 1: 1000}` :
  클래스 0을 100개, 클래스 1을 1000개로 샘플링합니다.
- `'majority'` :
  소수 클래스의 샘플 수를 다수 클래스의 샘플 수에
  맞추지 않고, 다수 클래스를 그대로 유지합니다.
- `'not minority'` :
  소수 클래스를 그대로 유지하고,
  다수 클래스의 샘플 수를 소수 클래스와

```

동일하게 맞춥니다.

요약

이 코드는 원본 데이터셋과 다양한 언더샘플링 전략에 따른 클래스 분포를 계산하여 `class\_cnt` 리스트에 저장하고, 이를 결합하여 표 형태로 출력합니다. 이를 통해 원본 데이터셋과 샘플링된 데이터셋의 클래스 분포를 비교할 수 있습니다.

```
class_cnt = list([pd.Series(y).value_counts()\
                  .rename('Original')])

for i in [{0: 100, 1: 1000}, 'majority', 'not minority']:
    rus = RandomUnderSampler(sampling_strategy=i)
    X_rus, y_rus = rus.fit_resample(X, y)
    class_cnt.append(
        pd.Series(y_rus).value_counts()\
        .rename('Sampling strategy: {}'.format(i))
    )

pd.concat(class_cnt, axis=1)
```

Out[21]:

	Original	Sampling strategy: {0: 100, 1: 1000}	Sampling strategy: majority	Sampling strategy: not minority
0	263	100	263	22
1	4715	1000	22	22
2	22	22	22	22

[Ex.8]

학습 데이터 df_train에서 대상 변수 Class를 제외한

모든 변수를 입력 변수로 설정합니다.

Class가 양성(Positive)인 비율을 아래와 같이 변화를 주어

df_train을 Random Under Sampling방법으로

샘플하고 성능의 차이를 살펴 봅니다.

샘플링 데이터로 학습하고, df_test로 성능을 측정합니다.

성능 지표는 f1-score 이고, 학습 모델은

로지스틱 회귀(Logistic Regression)입니다.

```
negative_ratio = [0.005, 0.01, 0.015, 0.02, 0.03, 0.05, 0.08, 0.1, 0.15]
```

```
In [22]: '''
코드는 언더샘플링 기법을 사용하여
데이터셋의 클래스 불균형 문제를 해결하고,
다양한 언더샘플링 비율에 따른 로지스틱 회귀 모델의
성능을 측정하는 작업을 수행합니다.
구체적으로, 각 언더샘플링 비율에 대한 모델의 훈련 및
테스트 성능(F1 스코어와 정확도)을 기록합니다.

### 데이터 및 모델 준비
- `f1_train_rus`, `f1_test_rus`:
  훈련 및 테스트 데이터에 대한
  F1 스코어를 저장하는 리스트.
- `acc_train`,
  `acc_test`: 훈련 및 테스트 데이터에 대한
  정확도를 저장하는 리스트.
- `X_cols`:
  'Class' 열을 제외한 데이터프레임의
  모든 열 이름을 저장하는 리스트.

### 모델 정의 및 원본 데이터셋으로 훈련
- `clf_lr`: 표준화 처리기(`StandardScaler()`)와
  로지스틱 회귀 모델(`LogisticRegression()`)을
  결합한 파이프라인.
- `clf_lr.fit(...)` : 원본 훈련 데이터셋을 사용하여
  모델을 훈련시킴.

### 원본 데이터셋에 대한 성능 측정
- `predict`:
```

```

    훈련 및 테스트 데이터에 대한 예측을 수행.
- `f1_score`, `accuracy_score`:
    예측 결과에 대한 F1 스코어와 정확도를 계산하여
    리스트에 추가.

### 다양한 언더샘플링 비율에 따른 성능 측정
- `negative_ratio`: 다양한 언더샘플링 비율 리스트.
- `RandomUnderSampler(sampling_strategy=i,
    random_state=123)`:
    지정된 언더샘플링 비율로 데이터셋을 언더샘플링.
- `fit_resample(...)`:
    지정된 비율에 따라 데이터셋을 샘플링.
- 언더샘플링된 데이터셋으로 모델을 훈련하고,
    훈련 및 테스트 데이터에 대한 성능을 측정하여
    리스트에 추가.

### 초기 클래스 비율을 리스트에 추가
- `df_train['Class'].value_counts(normalize=True).loc[1]`:
    원본 훈련 데이터셋에서 클래스 1의 비율을 계산.
- 이를 `negative_ratio` 리스트의 첫 번째 요소로 추가하여,
    원본 데이터셋의 클래스 비율도 포함시킴.

### 요약
코드는 다양한 언더샘플링 비율에 따른
로지스틱 회귀 모델의 성능을 측정합니다.
각 비율에 대해 모델을 훈련시키고,
훈련 및 테스트 데이터셋에서의 F1 스코어와 정확도를
계산하여 리스트에 저장합니다.
결과적으로, 원본 데이터셋과 다양한 언더샘플링
비율에 따른 모델 성능을 비교할 수 있습니다.
'''

from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import f1_score, accuracy_score

f1_train_rus, f1_test_rus = list(), list()
acc_train, acc_test = list(), list()
X_cols = [i for i in df_train.columns if i != 'Class']

```



```

# 표준화 처리기와 로지스틱 회귀모델을 결합합니다.
clf_lr = make_pipeline(StandardScaler(),
                        LogisticRegression(solver='lbfgs'))
clf_lr.fit(df_train[X_cols], df_train['Class'])

# 언더샘플링하지 않았을 경우의 성능을 기록합니다.
prd = clf_lr.predict(df_train[X_cols])
f1_train_rus.append(f1_score(df_train['Class'], prd))
acc_train.append(accuracy_score(df_train['Class'], prd))
prd = clf_lr.predict(df_test[X_cols])
f1_test_rus.append(f1_score(df_test['Class'], prd))
acc_test.append(accuracy_score(df_test['Class'], prd))

# 설정에 따른 성능을 측정하여 기록합니다.
negative_ratio
= [0.005, 0.01, 0.015, 0.02, 0.03, 0.05, 0.08, 0.1, 0.15]
for i in negative_ratio:
    rus = RandomUnderSampler(sampling_strategy=i,
                             random_state=123)

    X_rus, y_rus
        = rus.fit_resample(df_train[X_cols],
                           df_train['Class'])

    clf_lr
        = make_pipeline(StandardScaler(),
                        LogisticRegression(solver='lbfgs'))
    clf_lr.fit(X_rus, y_rus)

    prd = clf_lr.predict(X_rus)
    f1_train_rus.append(f1_score(y_rus, prd))
    acc_train.append(accuracy_score(y_rus, prd))

    prd = clf_lr.predict(df_test[X_cols].values)
    f1_test_rus.append(f1_score(df_test['Class'], prd))
    acc_test.append(accuracy_score(df_test['Class'], prd))

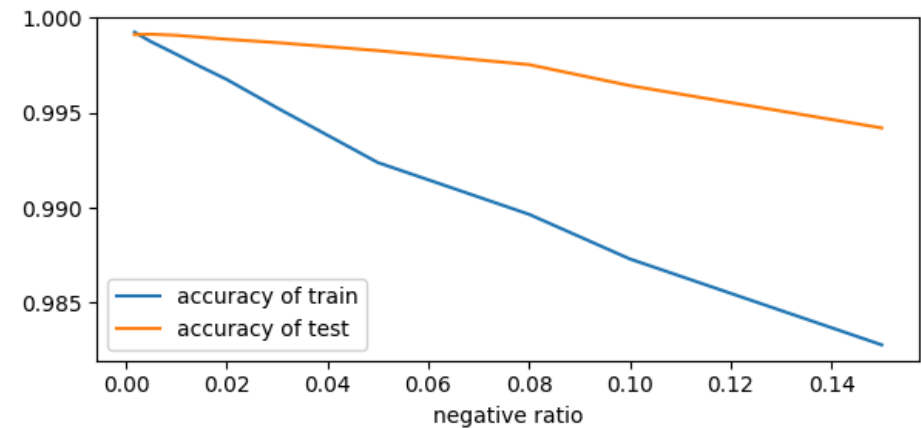
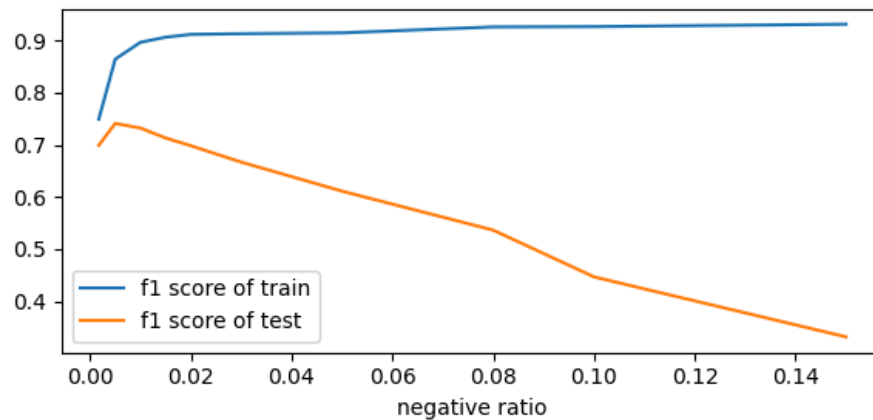
negative_ratio
= [df_train['Class'].value_counts(normalize=True).loc[1] \
  + negative_ratio

```

```
In [23]: fig, axes = plt.subplots(1, 2, figsize=(12, 3))
axes[0].plot(negative_ratio, f1_train_rus,
             label='f1 score of train')
axes[0].plot(negative_ratio, f1_test_rus,
             label='f1 score of test')
axes[0].set_xlabel('negative ratio')
axes[0].legend()

axes[1].plot(negative_ratio, acc_train,
             label='accuracy of train')
axes[1].plot(negative_ratio, acc_test,
             label='accuracy of test')
axes[1].set_xlabel('negative ratio')
axes[1].legend()

plt.tight_layout()
plt.show()
```



```
In [1]: '''
Tomek's Link 알고리즘은 데이터셋의 클래스 불균형 문제를
해결하기 위한 언더샘플링 기법 중 하나입니다.
Tomek's Link는 이웃간의 관계를 이용하여 데이터셋에서
노이즈를 제거하거나 클래스 간의 경계를 명확히 하여
모델의 성능을 향상시키는 방법입니다.

### Tomek's Link의 정의
```

Tomek's Link는 다음과 같이 정의됩니다:

- 두 샘플 $((x_i, y_i))$ 와 $((x_j, y_j))$ 가 서로의 가장 가까운 이웃일 때, $(y_i \neq y_j)$ 이면 이 두 샘플은 Tomek's Link를 형성합니다.

즉, 서로 다른 클래스에 속하는 두 샘플이 가장 가까운 이웃 관계에 있을 때, 이 둘을 연결하는 링크를 Tomek's Link라고 합니다.

Tomek's Link 알고리즘 절차

1. 이웃 탐색: 각 샘플의 가장 가까운 이웃을 찾습니다. 일반적으로 유클리드 거리 또는 다른 거리 척도를 사용하여 가장 가까운 이웃을 계산합니다.
2. 링크 탐색: 서로 다른 클래스에 속하는 샘플들 중에서 가장 가까운 이웃 관계에 있는 샘플들을 찾습니다. 이러한 샘플 쌍을 Tomek's Link라고 합니다.
3. 샘플 제거: Tomek's Link를 형성하는 샘플 중 하나 또는 둘 다 제거합니다. 주로 다수 클래스의 샘플을 제거하여 클래스 불균형을 완화합니다.

결과 해석

- 원본 데이터 클래스 분포: 클래스 0이 900개, 클래스 1이 100개.
- Tomek Links 적용 후 클래스 분포: 클래스 0의 샘플이 일부 제거되어 클래스 불균형이 완화됨.

Tomek's Link의 장점과 단점

장점

1. 노이즈 제거: 서로 다른 클래스의 가장 가까운 샘플을 제거함으로써 데이터의 노이즈를 줄일 수 있습니다.
2. 경계 명확화: 클래스 간의 경계를 명확히 하여 모델의 일반화 성능을 향상시킬 수 있습니다.
3. 간단한 구현: 구현이 비교적 간단하며, 데이터 전처리 과정에서 쉽게 적용할 수 있습니다.

단점

1. 데이터 손실: 다수 클래스의 샘플을 제거하기 때문에

중요한 정보를 잃을 수 있습니다.

2. 부분적 해결: 불균형이 심한 경우, Tomek's Link만으로는 클래스 불균형을 완전히 해결하기 어려울 수 있습니다.
3. 계산 비용: 모든 샘플에 대해 가장 가까운 이웃을 계산해야 하므로, 대규모 데이터셋에서는 계산 비용이 높아질 수 있습니다.

결론

Tomek's Link 알고리즘은 클래스 불균형 문제를 해결하는 데 유용한 도구입니다. 특히, 노이즈 제거와 클래스 경계 명확화에 효과적입니다. 그러나 데이터 손실을 초래할 수 있으므로, 다른 샘플링 기법과 결합하여 사용하는 것이 좋습니다. 데이터의 특성과 문제의 목적에 따라 적절한 방법을 선택하는 것이 중요합니다.

```
'''
import numpy as np
import pandas as pd
from sklearn.datasets import make_classification
from sklearn.neighbors import NearestNeighbors
from imblearn.under_sampling import TomekLinks

# 데이터 생성
X, y = make_classification(
    n_samples=1000, n_features=2, n_informative=2,
    n_redundant=0, n_clusters_per_class=1,
    weights=[0.9, 0.1],
    class_sep=0.8, random_state=123
)

# Tomek's Link를 사용한 언더샘플링
tl = TomekLinks()
X_res, y_res = tl.fit_resample(X, y)

# 원본 데이터와 샘플링 후 데이터 비교
print('원본 데이터 클래스 분포:\n',
      pd.Series(y).value_counts())
print('Tomek Links 적용 후 클래스 분포:\n',
      pd.Series(y_res).value_counts())
'''
```

원본 데이터 클래스 분포:

0 895

1 105

Name: count, dtype: int64

Tomek Links 적용 후 클래스 분포:

0 893

1 105

Name: count, dtype: int64

Tomek's Link 알고리즘

Tomek's Link

서로 다른 클래스에 속하는 두 점의 거리가

두 점에서의 가장 가까운 경우를 Tomek's Link라고 합니다.

Tomek's Link는 결정 경계면과 가깝게 위치하여

클래스 분류에 혼동을 일으킬 가능성이 높습니다.

Tomek's Link 중에서 둘 중에 하나의 클래스가

언더 샘플링 대상 클래스면 데이터셋에서 제외 시킵니다.

[Ex.9]

Tomek's Link 방법을 확인해 보고자, 3개 클래스를 지닌

클래스 불균형 문제를 지닌 2차원 데이터셋을 만듭니다.

Tomek's Link를 직접 구해 보고, 결과를 시각화해 봅니다.

In [24]:

```
'''
코드는 `make_blobs` 함수를 사용하여 2차원
데이터 샘플을 생성하고, 생성된 데이터를
`pandas` 데이터프레임으로 변환하여
레이블을 문자열로 설정하는 작업을 수행합니다.
```

1. `make\_blobs` 함수 사용

- ``n_samples``:
각 클러스터에 대해 생성할 샘플의 수를 지정합니다.
여기서는 세 개의 클러스터에 대해
각각 150개, 50개, 30개의 샘플을 생성합니다.
- ``centers``:
각 클러스터의 중심점을 지정합니다.
클러스터 중심은 ``[-0.5, 0]``, ``[2, 2]``,
``[1.5, -1]``입니다.
- ``cluster_std``:
각 클러스터의 표준 편차를 지정합니다.
여기서는 각각 0.5, 1.0, 0.7의 표준 편차를 갖습니다.
클러스터의 표준 편차는 클러스터가
퍼지는 정도를 나타냅니다.

``make_blobs`` 함수는 지정된 클러스터 개수와
중심점 및 표준 편차를 기반으로
샘플 데이터 ``X``와 레이블 ``y``를 생성합니다.

2. `pandas` 데이터프레임 생성 및 레이블 변환

- ``pd.DataFrame(X, columns=['X1', 'X2'])``:
``make_blobs``로 생성된 데이터 ``X``를 ``pandas``
데이터프레임으로 변환합니다.
여기서 ``X``는 2차원 데이터로,
두 개의 열(``X1``, ``X2``)로 구성됩니다.
- ``.assign(y = y.astype('str'))``:
``y`` 레이블을 데이터프레임에 추가하고,
이를 문자열로 변환합니다.
``astype('str')``는 정수형 레이블을
문자열로 변환합니다.

이 과정을 통해 생성된 데이터프레임
``df_tomek``은 다음과 같은 구조를 갖습니다:

- ``X1``, ``X2`` 열: 각 샘플의 2차원 좌표값
- ``y`` 열: 문자열로 변환된 레이블 값

예시

이 과정을 거친 후 데이터프레임 ``df_tomek``의
일부를 출력하면 다음과 비슷한 형태가 됩니다:

	X1	X2	y
0	0.0	0.0	0
1	1.0	1.0	1
2	2.0	2.0	2

```
|-----|-----|-----|
| -0.25 |  0.10 | '0' |
|  2.05 |  2.15 | '1' |
|  1.60 | -0.90 | '2' |
|   ... |   ... | ... |
```

- 여기서 각 행은 하나의 샘플을 나타내며,
 `x1`, `x2`는 해당 샘플의 좌표값, `y`는
 해당 샘플의 클러스터 레이블을 나타냅니다.
 ...

```
from sklearn.datasets import make_blobs
```

2차원 데이터 샘플 생성

```
X, y = make_blobs(
    n_samples=[150, 50, 30],
    centers=[[-0.5, 0], [2, 2], [1.5, -1]],
    cluster_std=[0.5, 1.0, 0.7], random_state=123
)
```

```
df_tomek = pd.DataFrame(X, columns=['X1', 'X2'])\
    .assign(y = y.astype('str'))
```

In [25]:

```
'''
코드는 Tomek 링크를 식별하기 위해 데이터 포인트 간의
거리 계산 및 최근접점을 찾는 과정을 수행합니다.
Tomek 링크는 분류 문제에서 오버샘플링이나 언더샘플링을
수행할 때 데이터의 균형을 맞추기 위해 자주 사용됩니다.
'''
```

1. 주요 변수 정의

```
- `X_cols`:
    좌표를 나타내는 열 이름을 리스트로 정의합니다.
    여기서는 `X1`과 `X2`입니다.
- `majority_class`:
    가장 많은 샘플을 가진 클래스를 식별합니다.
    `df_tomek['y'].value_counts()`는 각 클래스의
    샘플 수를 반환하고, `idxmax()`는 그 중 최대 값을
    가지는 클래스 레이블을 반환합니다.
```

2. 데이터 포인트 간 거리 계산

```
- `pdist(df_tomek[X_cols])`:
    `pdist` 함수는 데이터 포인트 간의 쌍별 거리를
    계산합니다. 여기서는 유클리드 거리를 계산합니다.
- `squareform`:
    `pdist`의 결과는 1차원 배열로 반환되는데,
    이를 거리 행렬 형식으로 변환합니다.
- `pd.DataFrame`:
    거리 행렬을 데이터프레임으로 변환하여,
    각 포인트 간의 거리를 쉽게 참조할 수 있도록 합니다.
    인덱스와 열 이름은 `df_tomek`의 인덱스와
    동일하게 설정됩니다.
```

3. 최근접점 인덱스 계산

```
- `df_dist.apply`:
    각 행마다 람다 함수를 적용합니다.
- `x.loc[x.index != x.name]`:
    자기 자신을 제외한 거리 값을 선택합니다.
- `idxmin()`:
    가장 작은 거리 값을 가지는 인덱스를 반환합니다.
    즉, 자기 자신을 제외한 최근접점의 인덱스를 찾습니다.
```

4. Tomek 링크 식별

```
- `(df_tomek['y'] == majority_class)`:
    다수 클래스에 속하는 데이터 포인트를 선택합니다.
- `(df_tomek['y'] != s_idx_nearest.map(df_tomek['y']))`:
    해당 데이터 포인트와 최근접점의 클래스가
    다른 경우를 선택합니다.
- `s_idx_nearest.map(s_idx_nearest)
    .pipe(lambda x: x.index == x)`:
    서로가 서로의 최근접점인 경우를 선택합니다.
    `s_idx_nearest`를 다시 매핑하여 두 포인트가
    서로의 최근접점인지 확인합니다.
- `loc`:
    위 조건들을 모두 만족하는 데이터 포인트의
    인덱스를 선택합니다.
```

전체 코드의 작동 방식 요약

1. 데이터 포인트 간의 거리 행렬을 계산합니다.
2. 각 데이터 포인트에 대해 가장 가까운
(자기 자신을 제외한) 인접점의 인덱스를 찾습니다.

3. 다수 클래스에 속하는 데이터 포인트 중에서 최근접점이 서로 다른 클래스이고, 서로의 최근접점인 데이터 포인트 쌍을 식별합니다.
4. 이러한 Tomek 링크의 인덱스를 반환합니다.

이를 통해 데이터셋에서 불균형을 제거하거나 줄이기 위해 사용할 수 있는 Tomek 링크를 식별할 수 있습니다.

```
from scipy.spatial.distance import pdist,squareform

X_cols = ['X1', 'X2']
majority_class = df_tomek['y'].value_counts().idxmax()

# 최근접점을 구하기 위해 데이터 포인트간 거리를 구합니다.
df_dist = pd.DataFrame(
    squareform(pdist(df_tomek[X_cols])),
    index = df_tomek.index,
    columns = df_tomek.index
)

# 데이터 포인트에서 자기자신을 제외한
# 최근접점의 인덱스를 구합니다.
s_idx_nearest = df_dist.apply(lambda x: \
                               x.loc[x.index != x.name].idxmin(),
                               axis=1)

tomek_links_index = s_idx_nearest.loc[
    # 다수 클래스 인지 확인
    (df_tomek['y'] == majority_class) &
    # 서로가 다른 클래스인지 확인
    (df_tomek['y'] != s_idx_nearest.map(df_tomek['y'])) &
    # 서로가 최근접점인지 확인
    s_idx_nearest.map(s_idx_nearest)\
    .pipe(lambda x: x.index == x)
].index.values

tomek_links_index
```

Out[25]: array([1, 4, 104, 186, 194], dtype=int64)

```
In [26]: '''
이 코드는 Tomek 링크를 시각화하고,
다수 클래스의 Tomek 링크를 제거한 후의
데이터를 시각화하는 작업을 수행합니다.

### 1. 서브플롯 생성
- `plt.subplots(1, 2, figsize=(12, 4))`:
    1행 2열의 서브플롯을 생성합니다.
    전체 그림의 크기는 가로 12인치, 세로 4인치로 설정됩니다.
- `fig, axes`:
    `fig`는 전체 그림(Figure) 객체,
    `axes`는 각 서브플롯(Axes) 객체를 가리킵니다.
    `axes`는 길이 2의 배열입니다.

### 2. 첫 번째 서브플롯 - Tomek 링크 시각화
- `sns.scatterplot`:
    `seaborn` 라이브러리를 사용하여 산점도를 그립니다.
- `data=df_tomek`:
    `df_tomek` 데이터프레임을 사용합니다.
- `x='X1', y='X2'`:
    `X1`과 `X2` 열을 사용하여 x축과 y축에 값을 매핑합니다.
- `hue='y'`:
    레이블 `y`에 따라 점의 색을 다르게 합니다.
- `s=20`: 점의 크기를 설정합니다.
- `ax=axes[0]`: 첫 번째 서브플롯에 그래프를 그립니다.
- `axes[0].set_title`, `axes[0].set_xlabel`, `axes[0].set_ylabel`:
    첫 번째 서브플롯의 제목과 축 레이블을 설정합니다.
- `axes[0].scatter`:
    첫 번째 서브플롯에 추가적인 점을 그립니다.
- `x = df_tomek.loc[tomek_links_index, 'X1']`:
    Tomek 링크의 `X1` 좌표를 사용합니다.
- `y = df_tomek.loc[tomek_links_index, 'X2']`:
    Tomek 링크의 `X2` 좌표를 사용합니다.
- `s = 50`: 점의 크기를 50으로 설정합니다.
- `color='r', facecolors='none':
    점의 색상을 빨간색 테두리로 설정하고,
    내부는 채우지 않습니다.

### 3. 두 번째 서브플롯
- 다수 클래스의 Tomek 링크 제거 후 시각화
```

```
- `sns.scatterplot`:
    `seaborn` 라이브러리를 사용하여 산점도를 그립니다.
- `data=df_tomek.loc[~df_tomek.index.isin(tomek_links_index)]`:
    Tomek 링크에 해당하는 인덱스를 제외한 데이터만 사용합니다.
- `x='X1', y='X2'`:
    `X1`과 `X2` 열을 사용하여 x축과 y축에 값을 매핑합니다.
- `hue='y'`:
    레이블 `y`에 따라 점의 색을 다르게 합니다.
- `s=20`: 점의 크기를 설정합니다.
- `ax=axes[1]`: 두 번째 서브플롯에 그래프를 그립니다.
- `axes[1].set_title`, `axes[1].set_xlabel`, `axes[1].set_ylabel`:
    두 번째 서브플롯의 제목과 축 레이블을 설정합니다.
```

전체 코드 요약

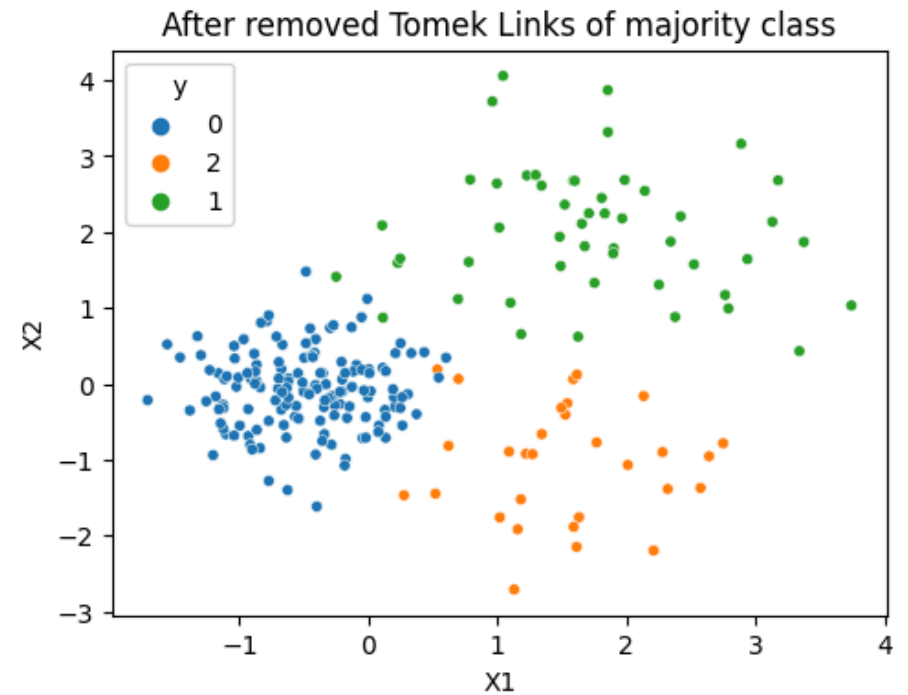
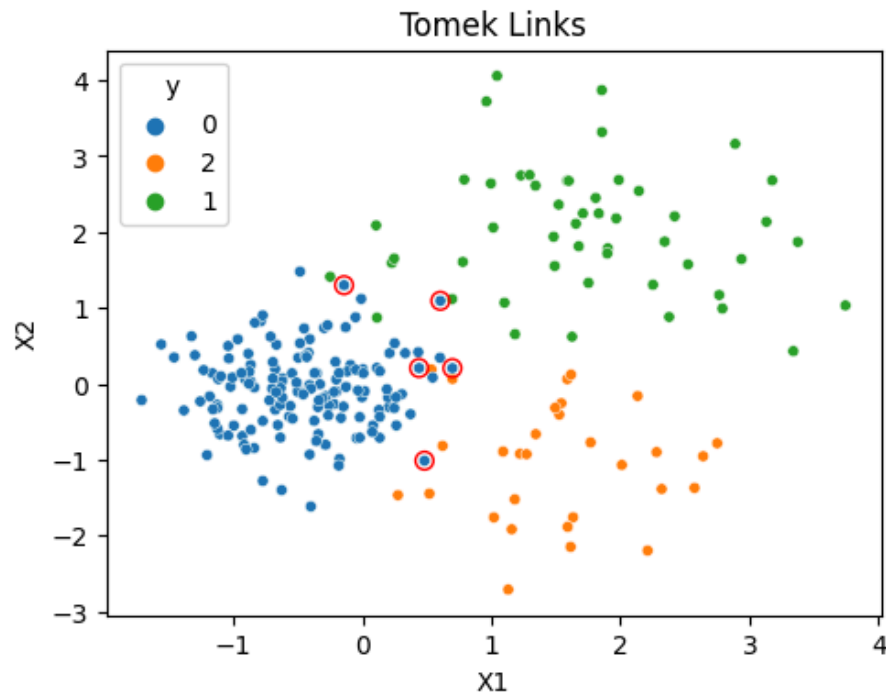
```
- 두 개의 서브플롯을 생성합니다.
- 첫 번째 서브플롯에는 전체 데이터 포인트와
    Tomek 링크에 해당하는 포인트를 빨간색 테두리로 표시합니다.
- 두 번째 서브플롯에는 다수 클래스의
    Tomek 링크를 제거한 후의 데이터를 시각화합니다.
- 이를 통해 Tomek 링크가 어떤 데이터 포인트를
    제거할지를 시각적으로 확인할 수 있습니다.
...
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.scatterplot(data=df_tomek, x='X1', y='X2',
                hue='y', s=20, ax=axes[0])
axes[0].set_title('Tomek Links')
axes[0].set_xlabel('X1')
axes[0].set_ylabel('X2')

axes[0].scatter(
    x = df_tomek.loc[tomek_links_index, 'X1'],
    y = df_tomek.loc[tomek_links_index, 'X2'],
    s = 50, color='r', facecolors='none'
)

sns.scatterplot(
    data=df_tomek.loc[~df_tomek.index.isin(tomek_links_index)],
    x='X1', y='X2', hue='y', s=20, ax=axes[1]
)
```

```
axes[1].set_title('After removed Tomek Links of majority class')
axes[1].set_xlabel('X1')
axes[1].set_ylabel('X2')
plt.show()
```



imblearn.under_sampling.TomekLinks

TomekLinks는 TomekLink의 수에 따라 샘플링 데이터 수가 정해지므로

임의의 개수로 표본 수를 만드는 것은 지원하지 않습니다.

어떤 클래스를 TomekLink로 지정할 지를 설정합니다.

sampling_strategy:

'majority': 가장 빈도가 높은 클래스만 리샘플링합니다.

'not minority': 가장 빈도가 낮은 클래스를 제외하고 리샘플링

'not majority': 가장 빈도가 높은 클래스를 제외하고 리샘플링

'auto': 자동 - not minority

```
In [27]: from imblearn.under_sampling import TomekLinks

tomek = TomekLinks(sampling_strategy='majority')
X_tomek, y_tomek
        = tomek.fit_resample(df_tomek[X_cols], df_tomek['y'])
```

[Ex.10]

학습 데이터 df_train에서 대상 변수 Class를 제외한 모든 변수를 입력 변수로 설정합니다.

Class가 양성(Positive)인 비율을 Tomek's Link 알고리즘으로 Under sampling 합니다.

샘플링 데이터로 학습하고, df_test로 성능을 측정합니다.

성능 지표는 f1-score 이고, 학습 모델은 로지스틱 회귀(Logistic Regression)입니다.

```
In [28]: '''
코드는 Tomek 링크를 사용하여 다수 클래스를
언더샘플링하고, 로지스틱 회귀 모델을 학습 및
평가하는 과정을 반복하여 F1 점수와 정확도를 기록합니다.

### 1. 변수 초기화 및 로지스틱 회귀 모델 학습
- `f1_train_tomek`, `f1_test_tomek`:
    각 반복에서 훈련 및 테스트 데이터의
    F1 점수를 저장할 리스트를 초기화합니다.
- `acc_train`, `acc_test`:
    각 반복에서 훈련 및 테스트 데이터의
    정확도를 저장할 리스트를 초기화합니다.
- `X_cols`:
    훈련 데이터의 입력 변수(피처) 이름 리스트를
    생성합니다. 여기서 `Class` 열을 제외합니다.
- `make_pipeline(StandardScaler(),
    LogisticRegression(solver='lbfgs'))`:
    파이프라인을 생성합니다. 먼저 `StandardScaler`를 사용하여
```

```

    데이터를 표준화한 후, 로지스틱 회귀 모델을 적용합니다.
- `clf_lr.fit(df_train[X_cols], df_train['Class'])`:
    파이프라인을 사용하여 훈련 데이터로 모델을 학습합니다.

### 2. Tomek 링크를 사용한 언더샘플링 및 모델 학습
- `TomekLinks(sampling_strategy='majority')`:
    Tomek 링크 객체를 생성합니다.
    `sampling_strategy='majority'`는 다수 클래스를
    언더샘플링하는 전략을 사용함을 의미합니다.
- `rus.fit_resample(df_train[X_cols], df_train['Class'])`:
    Tomek 링크를 사용하여 훈련 데이터를 재샘플링합니다.
    다수 클래스의 Tomek 링크에 해당하는 샘플이
    제거된 데이터(`X_tomek`, `y_tomek`)가 반환됩니다.
- `make_pipeline(StandardScaler(),
    LogisticRegression(solver='lbfgs'))`:
    파이프라인을 다시 생성합니다.
- `clf_lr.fit(X_tomek, y_tomek)`:
    재샘플링된 데이터를 사용하여 파이프라인을 학습합니다.

### 3. 모델 평가 및 결과 저장
- `prd = clf_lr.predict(X_tomek)`:
    재샘플링된 훈련 데이터에 대해 예측을 수행합니다.
- `f1_train_tomek.append(f1_score(y_tomek, prd))`:
    예측 결과에 대한 F1 점수를 계산하고 리스트에 추가합니다.
- `acc_train.append(accuracy_score(y_tomek, prd))`:
    예측 결과에 대한 정확도를 계산하고 리스트에 추가합니다.

- `prd = clf_lr.predict(df_test[X_cols].values)`:
    테스트 데이터에 대해 예측을 수행합니다.
- `f1_test_tomek.append(f1_score(df_test['Class'], prd))`:
    예측 결과에 대한 F1 점수를 계산하고 리스트에 추가합니다.
- `acc_test.append(accuracy_score(df_test['Class'], prd))`:
    예측 결과에 대한 정확도를 계산하고 리스트에 추가합니다.

### 4. 음수 클래스 비율 기록
- `df_train['Class'].value_counts(normalize=True).loc[1]`:
    훈련 데이터에서 클래스 `1`의 비율을 계산합니다.
- `negative_ratio = [df_train['Class'].value_counts(normalize=True)
    .loc[1]] + negative_ratio`:
    계산된 비율을 `negative_ratio` 리스트의 앞에 추가합니다.

```

전체 코드의 작동 요약

1. F1 점수와 정확도를 기록할 리스트를 초기화합니다.
2. 로지스틱 회귀 모델을 학습합니다.
3. Tomek 링크를 사용하여 다수 클래스를 언더샘플링합니다.
4. 재샘플링된 데이터를 사용하여 모델을 다시 학습합니다.
5. 재샘플링된 훈련 데이터와 원래의 테스트 데이터에 대해 모델을 평가하고, 결과를 기록합니다.
6. 음수 클래스(소수 클래스) 비율을 기록합니다.

이를 통해 Tomek 링크를 사용한 데이터 전처리가
모델 성능에 미치는 영향을 평가할 수 있습니다.
'''

```
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import f1_score, accuracy_score

f1_train_tomek, f1_test_tomek = list(), list()
acc_train, acc_test = list(), list()
X_cols = [i for i in df_train.columns if i != 'Class']
clf_lr = make_pipeline(StandardScaler(), \
                        LogisticRegression(solver='lbfgs'))
clf_lr.fit(df_train[X_cols], \
           df_train['Class'])

rus = \
    = TomekLinks(sampling_strategy='majority')
X_tomek, y_tomek = \
    = rus.fit_resample(df_train[X_cols],
                       df_train['Class'])
clf_lr = \
    = make_pipeline(StandardScaler(),
                    LogisticRegression(solver='lbfgs'))
clf_lr.fit(X_tomek, y_tomek)

prd = clf_lr.predict(X_tomek)
f1_train_tomek.append(f1_score(y_tomek, prd))
acc_train.append(accuracy_score(y_tomek, prd))
```

```

prd = clf_lr.predict(df_test[X_cols].values)
f1_test_tomek.append(f1_score(df_test['Class'], prd))
acc_test.append(accuracy_score(df_test['Class'], prd))

negative_ratio \
    = [df_train['Class'].value_counts(normalize=True).loc[1]] \
      + negative_ratio

```

In [29]: f1_train_tomek, f1_test_tomek

Out[29]: ([0.7478005865102639], [0.6987951807228915])

언더 샘플링의 장단점

장점

- 데이터셋의 크기를 줄이기 때문에

모델 학습이 더 빠르게 진행될 수 있습니다.

- 잡음 감소: 다수 클래스의 샘플을 제거함으로써

데이터셋의 잡음감소 효과를 얻을 수 있습니다.

단점

- 정보 손실: 다수 클래스의 샘플을 제거하면

데이터의 다양성이 감소할 수 있습니다.

- 소수 클래스의 대표성 손실: 중요한 소수 클래스의 정보가

대조군이 되는 다수 클래스의 표본이 사라짐에 따라

정보가 손실이 될 수 있습니다.

샘플링 알고리즘 별 특징 비교


```
In [43]: '''
코드는 네 가지 샘플링 기법 (Random Oversampling, SMOTE,
ADASYN, Random Undersampling)에 대한 테스트 데이터의 F1 점수를
시각화합니다. 각 기법의 성능을 비교하기 위해 서브플롯을
생성하고, 각 서브플롯에 해당 기법의 F1 점수를 플로팅합니다.

### 1. 서브플롯 생성
- `plt.subplots(1, 4, figsize=(16, 3))`:
    1행 4열의 서브플롯을 생성합니다.
    전체 그림의 크기는 가로 16인치, 세로 3인치로 설정됩니다.
- `fig, axes`:
    `fig`는 전체 그림(Figure) 객체, `axes`는
    각 서브플롯(Axes) 객체를 가리킵니다.
    `axes`는 길이 4의 배열입니다.

### 2. 각 샘플링 기법에 대한 F1 점수 플로팅
- `for ax, i, dat in zip(axes, [...], [...])`:
    서브플롯, 샘플링 기법 이름, 각 기법의
    F1 점수를 동시에 반복합니다.
- `ax`: 현재 서브플롯(Axes) 객체.
- `i`: 현재 샘플링 기법 이름.
- `dat`: 현재 샘플링 기법의 F1 점수 리스트.
- `if i == 'Random Undersampling':`:
    Random Undersampling의 경우.
- `ax.plot(negative_ratio[1:], dat, label=i)`:
    음수 클래스의 비율 (`negative_ratio[1:]`)에 대해
    F1 점수 (`dat`)를 플로팅합니다.
    `label`에는 샘플링 기법 이름을 설정합니다.
- `else`: 나머지 경우 (Random Oversampling, SMOTE, ADASYN).
- `ax.plot(postive_ratio, dat, label=i)`:
    양수 클래스의 비율 (`postive_ratio`)에 대해
    F1 점수 (`dat`)를 플로팅합니다.
    `label`에는 샘플링 기법 이름을 설정합니다.
- `ax.set_title(i)`:
    서브플롯의 제목을 샘플링 기법 이름으로 설정합니다.

### 3. 레이아웃 조정 및 그래프 출력
- `plt.tight_layout()`:
    서브플롯 간의 간격을 자동으로 조정하여
    레이아웃을 최적화합니다.
```

그래프들이 서로 겹치지 않도록 합니다.

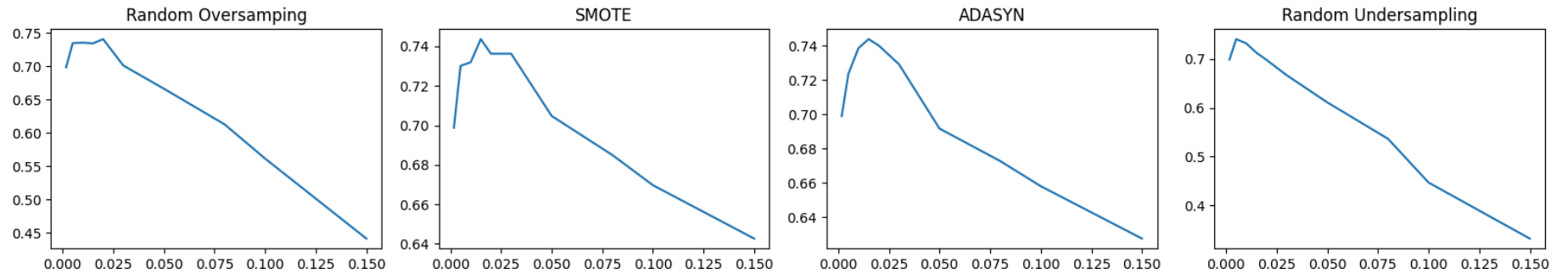
전체 코드의 작동 요약

1. 1행 4열의 서브플롯을 생성하여
네 가지 샘플링 기법에 대한 서브플롯을 준비합니다.
2. 각 서브플롯에 대해 해당 샘플링 기법의
테스트 데이터 F1 점수를 플로팅합니다.
 - Random Undersampling의 경우
음수 클래스 비율 (`negative\_ratio[1:]`)에 대해
F1 점수를 플로팅합니다.
 - 나머지 기법들의 경우 양수 클래스 비율 (`positive\_ratio`)에
대해 F1 점수를 플로팅합니다.
3. 각 서브플롯의 제목을 샘플링 기법 이름으로 설정합니다.
4. 레이아웃을 최적화하여 서브플롯 간의 간격을 조정합니다.

이를 통해 네 가지 샘플링 기법의 성능을
시각적으로 비교할 수 있습니다.

'''

```
fig, axes = plt.subplots(1, 4, figsize=(16, 3))
for ax, i, dat in zip(axes,
                      ['Random Oversampling', 'SMOTE',
                       'ADASYN', 'Random Undersampling'],
                      [f1_test_ros, f1_test_smote, f1_test_adasyn, f1_test_rus]):
    if i == 'Random Undersampling':
        ax.plot(negative_ratio[1:], dat, label=i)
    else:
        ax.plot(positive_ratio, dat, label=i)
    ax.set_title(i)
plt.tight_layout()
plt.show()
```



In [30]:

```
'''
코드는 각 샘플링 기법(Random Oversampling, SMOTE, ADASYN,
Random Undersampling, Tomek Links)에서 테스트 데이터에 대한
F1 점수의 최댓값을 계산하고 출력합니다.

### 코드 설명

각 샘플링 기법에 대해 F1 점수의 최댓값을
계산하는 방법은 다음과 같습니다:

1. `np.max(f1_test_ros)` :
   - `f1_test_ros` 리스트에서 최댓값을 계산합니다.
   - `f1_test_ros`는 Random Oversampling 기법을
     사용한 경우의 테스트 데이터에 대한 F1 점수 리스트입니다.

2. `np.max(f1_test_smote)` :
   - `f1_test_smote` 리스트에서 최댓값을 계산합니다.
   - `f1_test_smote`는 SMOTE 기법을 사용한 경우의
     테스트 데이터에 대한 F1 점수 리스트입니다.

3. `np.max(f1_test_adasyn)` :
   - `f1_test_adasyn` 리스트에서 최댓값을 계산합니다.
   - `f1_test_adasyn`는 ADASYN 기법을 사용한 경우의
     테스트 데이터에 대한 F1 점수 리스트입니다.

4. `np.max(f1_test_rus)` :
   - `f1_test_rus` 리스트에서 최댓값을 계산합니다.
   - `f1_test_rus`는 Random Undersampling 기법을 사용한
     경우의 테스트 데이터에 대한 F1 점수 리스트입니다.

5. `np.max(f1_test_tomek)` :
   - `f1_test_tomek` 리스트에서 최댓값을 계산합니다.
   - `f1_test_tomek`는 Tomek Links 기법을 사용한 경우의
     테스트 데이터에 대한 F1 점수 리스트입니다.
'''
```

실행 결과

위 코드를 실행하면, 각 샘플링 기법에서 테스트 데이터에 대한 F1 점수의 최댓값이 각각 출력됩니다.

이를 통해 각 기법이 얼마나 잘 작동했는지에 대한 비교를 할 수 있습니다.

이 결과는 Random Oversampling, SMOTE, ADASYN, Random Undersampling, Tomek Links 기법 각각에서 테스트 데이터에 대해 얻은 F1 점수의 최댓값을 나타냅니다. 이 값을 통해 어느 기법이 가장 성능이 좋았는지 비교할 수 있습니다.

```
np.max(f1_test_ros), np.max(f1_test_smote), np.max(f1_test_adasyn),
np.max(f1_test_rus), np.max(f1_test_tomek)
```

```
Out[30]: (0.7411167512690355,
          0.743718592964824,
          0.743718592964824,
          0.7407407407407408,
          0.6987951807228915)
```

3. 클래스 분류 임계점 조정

클래스를 분류하는 성능 지표를 최대화하는 임계점을 찾는 방법입니다.

```
In [2]: Image('152.png')
```

Out[2]: **[Ex.11]**

학습 데이터 df_train에서 대상 변수 Class를 제외한 모든 변수를 입력 변수로 설정합니다.

df_train으로 로지스틱 회귀 모델을 학습시킵니다.

양성으로 분류를 다음과 같이 정합니다.

$P(Class = True) > \alpha$, $P(Class = True)$ 는 양성으로 예측한 확률입니다.

df_train의 각 데이터의 양성일 확률 구하고, α 를 0.01 에서 0.99 까지의 0.01 간격으로 성능을 측정해봅니다.

가장 좋은 f1 score를 보인 α 를 찾습니다.

df_test의 예측 확률을 구해 이 α 로 양성 클래스를 분류하여 f1 score를 측정해봅니다.

```
In [31]: '''
코드는 로지스틱 회귀 모델을 학습하고, 훈련 데이터에
대한 예측 확률을 계산하는 과정을 수행합니다.

### 1. 리스트 초기화
- `f1_train_tomek`, `f1_test_tomek`:
    각 반복에서 훈련 및 테스트 데이터의
    F1 점수를 저장할 리스트를 초기화합니다.
- `acc_train`, `acc_test`:
    각 반복에서 훈련 및 테스트 데이터의
    정확도를 저장할 리스트를 초기화합니다.

### 2. 입력 변수 리스트 생성
- `X_cols`:
    훈련 데이터의 입력 변수(피처) 이름 리스트를 생성합니다.
    여기서 `Class` 열을 제외합니다.
    즉, `Class` 열은 레이블(타겟)로 사용되고,
    나머지 열들은 모델의 입력 피처로 사용됩니다.

### 3. 파이프라인 생성 및 모델 학습
```

```
- `make_pipeline(StandardScaler(),
    LogisticRegression(solver='lbfgs'))`:
    파이프라인을 생성합니다.
- `StandardScaler()`: 데이터를 표준화합니다
    (각 피처의 평균을 0, 표준편차를 1로 변환).
- `LogisticRegression(solver='lbfgs')`:
    로지스틱 회귀 모델을 사용합니다.
    `solver='lbfgs'`는 옵티마이저
    알고리즘을 지정합니다 (LBFGS 알고리즘).
- `clf_lr.fit(df_train[X_cols], df_train['Class'])`:
    파이프라인을 사용하여 훈련 데이터로 모델을 학습합니다.
- `df_train[X_cols]`: 입력 피처 데이터.
- `df_train['Class']`: 레이블 데이터.
```

4. 예측 확률 계산

```
- `clf_lr.predict_proba(df_train[X_cols])`:
    훈련 데이터에 대해 각 클래스에 대한 예측 확률을
    계산합니다. 이 함수는 (n_samples, n_classes) 형태의
    배열을 반환합니다.
- 각 샘플에 대해 두 개의 확률이 반환됩니다
    (클래스 0과 클래스 1에 대한 확률).
- `[:, 1]`: 모든 샘플에 대해 클래스 1에 대한 확률만 추출합니다.
```

전체 코드 요약

1. F1 점수와 정확도를 저장할 리스트를 초기화합니다.
2. 훈련 데이터의 입력 피처 이름 리스트를 생성합니다
 (``Class`` 열은 제외).
3. 파이프라인을 생성합니다. 파이프라인은 데이터를 표준화한 후
 로지스틱 회귀 모델을 적용합니다.
4. 훈련 데이터로 모델을 학습합니다.
5. 훈련 데이터에 대해 클래스 1에 대한 예측 확률을 계산합니다.

코드를 통해 학습된 로지스틱 회귀 모델은 훈련 데이터의
 각 샘플이 클래스 1일 확률을 출력하게 됩니다.
 이 확률은 이후에 모델의 성능을 평가하거나
 추가적인 분석에 사용할 수 있습니다.

```
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
```

```

from sklearn.metrics import f1_score, accuracy_score

f1_train_tomek, f1_test_tomek = list(), list()
acc_train, acc_test = list(), list()
X_cols = [i for i in df_train.columns if i != 'Class']
clf_lr = make_pipeline(StandardScaler(),
                        LogisticRegression(solver='lbfgs'))
clf_lr.fit(df_train[X_cols], df_train['Class'])

prob = clf_lr.predict_proba(df_train[X_cols])[:, 1]

```

```

In [32]: '''
이 코드는 예측 확률을 기반으로 다양한 임계값(threshold)을 적용하여
훈련 데이터의 F1 점수를 계산하고, 최적의 F1 점수를 달성하는
임계값을 찾는 과정을 수행합니다.

### 1. 임계값 배열 생성
- `np.linspace(0.01, 0.99, 101)` :
    0.01에서 0.99까지의 범위에서 101개의 균등한 값을 생성합니다.
- `thresholds` :
    이 배열은 다양한 임계값을 나타내며,
    각 임계값은 0.01 간격으로 증가합니다.
    즉, 예측 확률이 해당 임계값 이상일 때
    해당 샘플을 클래스 1로 분류하게 됩니다.

### 2. F1 점수 계산
- 리스트 내포를 사용하여 각 임계값에 대해 F1 점수를 계산합니다.
    - `for threshold in thresholds` : 각 임계값에 대해 반복합니다.
    - `f1_score(df_train['Class'], prob >= threshold)` :
        - `df_train['Class']` : 실제 레이블(타겟)입니다.
        - `prob >= threshold` :
            예측 확률(`prob`)이 현재 임계값보다 크거나 같으면 1,
            그렇지 않으면 0으로 이진화합니다.
            이 이진화된 값은 예측 클래스입니다.
    - `f1_score` :
        실제 레이블과 예측된 클래스를 비교하여 F1 점수를 계산합니다.
- `f1_scores` : 각 임계값에 대한 F1 점수를 저장하는 리스트입니다.

### 3. 최적의 F1 점수와 해당 임계값 찾기
- `np.argmax(f1_scores)` :

```

```

`f1_scores` 리스트에서 최댓값의 인덱스를 반환합니다.
즉, 가장 높은 F1 점수를 달성한 임계값의 인덱스를 찾습니다.
- `idx_max`: 최적의 F1 점수를 달성한 임계값의 인덱스입니다.

```

```

### 4. 최적의 F1 점수와 임계값 출력
- `f1_scores[idx_max]`: 최적의 F1 점수.
- `thresholds[idx_max]`: 최적의 F1 점수를 달성한 임계값.

```

```

### 전체 코드 요약
1. 임계값 배열 생성: 0.01에서 0.99까지
   101개의 균등한 임계값을 생성합니다.
2. F1 점수 계산: 각 임계값에 대해
   예측 확률을 이진화하여 F1 점수를 계산합니다.
3. 최적의 F1 점수와 임계값 찾기:
   가장 높은 F1 점수를 달성한 임계값을 찾습니다.
4. 최적의 F1 점수와 임계값 출력:
   최적의 F1 점수와 해당 임계값을 출력합니다.

```

```

이를 통해 모델의 성능을 최적화하는 임계값을 찾을 수 있습니다.
예측 확률을 다양한 임계값으로 이진화하여
F1 점수를 비교함으로써 최적의 분류 기준을 결정합니다.
'''

```

```

thresholds = np.linspace(0.01, 0.99, 101)
f1_scores = [f1_score(df_train['Class'], prob >= threshold )
              for threshold in thresholds]
idx_max = np.argmax(f1_scores)
f1_scores[idx_max], thresholds[idx_max]

```

Out[32]: (0.80970625798212, 0.11779999999999999)

```

In [33]: '''
코드는 이전 단계에서 찾은 최적의 임계값을 사용하여
테스트 데이터에 대한 F1 점수를 계산하는 과정입니다.

### 1. 테스트 데이터에 대한 예측 확률 계산
- `clf_lr.predict_proba(df_test[X_cols])`:
   테스트 데이터에 대해 각 클래스에 대한 예측 확률을 계산합니다.
   이 함수는 (n_samples, n_classes) 형태의 배열을 반환합니다.
- `df_test[X_cols]`: 테스트 데이터의 입력 피처입니다.

```



```

- 결과 배열에서 각 샘플에 대해 두 개의 확률이 반환됩니다
  (클래스 0과 클래스 1에 대한 확률).
- `[:, 1]`: 모든 샘플에 대해 클래스 1에 대한 확률만 추출합니다.
- `prob_test`:
  테스트 데이터에서 각 샘플이 클래스 1일 확률입니다.

### 2. 테스트 데이터에 대한 F1 점수 계산
- `df_test['Class']`:
  테스트 데이터의 실제 레이블(타겟)입니다.
- `prob_test >= thresholds[idx_max]`:
  최적의 임계값(`thresholds[idx_max]`)을 사용하여
  예측 확률을 이진화합니다.
- `prob_test >= thresholds[idx_max]`:
  예측 확률(`prob_test`)이 최적의 임계값보다
  크거나 같으면 1, 그렇지 않으면 0으로 이진화합니다.
  이 이진화된 값은 예측된 클래스입니다.
- `f1_score(df_test['Class'], prob_test >= thresholds[idx_max])`:
  실제 레이블과 예측된 클래스를 비교하여 F1 점수를 계산합니다.

### 전체 코드 요약
1. 테스트 데이터에 대한 예측 확률 계산:
  로지스틱 회귀 모델을 사용하여 테스트 데이터의
  각 샘플이 클래스 1일 확률을 계산합니다.
2. F1 점수 계산: 이전 단계에서 찾은 최적의 임계값을 사용하여
  테스트 데이터에 대한 예측 확률을 이진화하고,
  실제 레이블과 비교하여 F1 점수를 계산합니다.

이를 통해 테스트 데이터에 대한 모델의 성능을 평가할 수 있습니다.
이 과정은 모델이 새로운 데이터에 대해
얼마나 잘 일반화되는지를 확인하는 중요한 단계입니다.

### 예시 결과
만약 `thresholds[idx_max]`가 0.45라고 가정하면,
`prob_test >= 0.45`는 테스트 데이터의 각 샘플에 대해
예측 확률이 0.45 이상인 경우 클래스 1로 분류하고,
그렇지 않은 경우 클래스 0으로 분류합니다.
그 후, 실제 레이블(`df_test['Class']`)과 비교하여
F1 점수를 계산합니다. 이 F1 점수는 테스트 데이터에서
모델의 성능을 나타냅니다.
'''

```

```
prob_test = clf_lr.predict_proba(df_test[X_cols])[:, 1]
f1_score(df_test['Class'], prob_test >= thresholds[idx_max])
```

Out[33]: 0.7434554973821991

```
In [34]: '''
코드는 다양한 임계값에 따른 훈련 데이터와 테스트 데이터의
F1 점수를 시각화합니다. 최적의 임계값을 강조하고,
훈련 데이터와 테스트 데이터의 성능 차이를
비교할 수 있게 합니다.

### 1. 플롯 생성 및 크기 설정
- `plt.figure(figsize=(8, 3))`:
    가로 8인치, 세로 3인치의 새로운 그림(Figure)을 생성합니다.

### 2. 훈련 데이터의 F1 점수 플롯
- `plt.plot(thresholds, f1_scores, label='f1 score of train')`:
    - `thresholds`: x축 값, 다양한 임계값 배열.
    - `f1_scores`:
        y축 값, 각 임계값에 따른 훈련 데이터의 F1 점수.
    - `label='f1 score of train'`:
        훈련 데이터 F1 점수 플롯의 레이블을 설정합니다.

### 3. 최적의 임계값 강조
- `plt.axvline(thresholds[idx_max])`:
    - `thresholds[idx_max]`: 최적의 F1 점수를 달성한 임계값.
    - `plt.axvline`: 최적의 임계값에서 수직선을 그려 강조합니다.

### 4. 테스트 데이터의 F1 점수 플롯
- `plt.plot(thresholds, [f1_score(df_test['Class'],
    prob_test >= threshold) for threshold in thresholds],
    label='f1 score of test')`:
    - `thresholds`: x축 값, 다양한 임계값 배열.
    - `[f1_score(df_test['Class'], prob_test >= threshold)
        for threshold in thresholds]`:
        y축 값, 각 임계값에 따른 테스트 데이터의 F1 점수.
    - 리스트 내포를 사용하여 각 임계값에 대해
        F1 점수를 계산합니다.
    - `f1_score(df_test['Class'], prob_test >= threshold)`:
        테스트 데이터의 실제 레이블(`df_test['Class']`)과
```

```

    예측된 클래스(`prob_test >= threshold`)를 비교하여
    F1 점수를 계산합니다.
- `label='f1 score of test':`
    테스트 데이터 F1 점수 플롯의 레이블을 설정합니다.

### 5. 범례 추가
plt.legend()
- `plt.legend():`
    플롯에 범례를 추가하여 각 선이 무엇을
    나타내는지 설명합니다.

### 전체 코드 요약
1. 플롯 생성 및 크기 설정:
    가로 8인치, 세로 3인치의 새로운 플롯을 생성합니다.
2. 훈련 데이터의 F1 점수 플롯:
    다양한 임계값에 따른 훈련 데이터의
    F1 점수를 플롯합니다.
3. 최적의 임계값 강조:
    최적의 F1 점수를 달성한 임계값에서
    수직선을 그려 강조합니다.
4. 테스트 데이터의 F1 점수 플롯:
    다양한 임계값에 따른 테스트 데이터의
    F1 점수를 플롯합니다.
5. 범례 추가:
    플롯에 범례를 추가하여
    각 선의 의미를 설명합니다.
6. 플롯 표시: 플롯을 화면에 표시합니다.

이 시각화를 통해 다양한 임계값에 따른 훈련 데이터와
테스트 데이터의 F1 점수를 비교하고,
최적의 임계값이 어디에 있는지,
그리고 훈련 데이터와 테스트 데이터에서의
성능 차이를 쉽게 확인할 수 있습니다.
'''

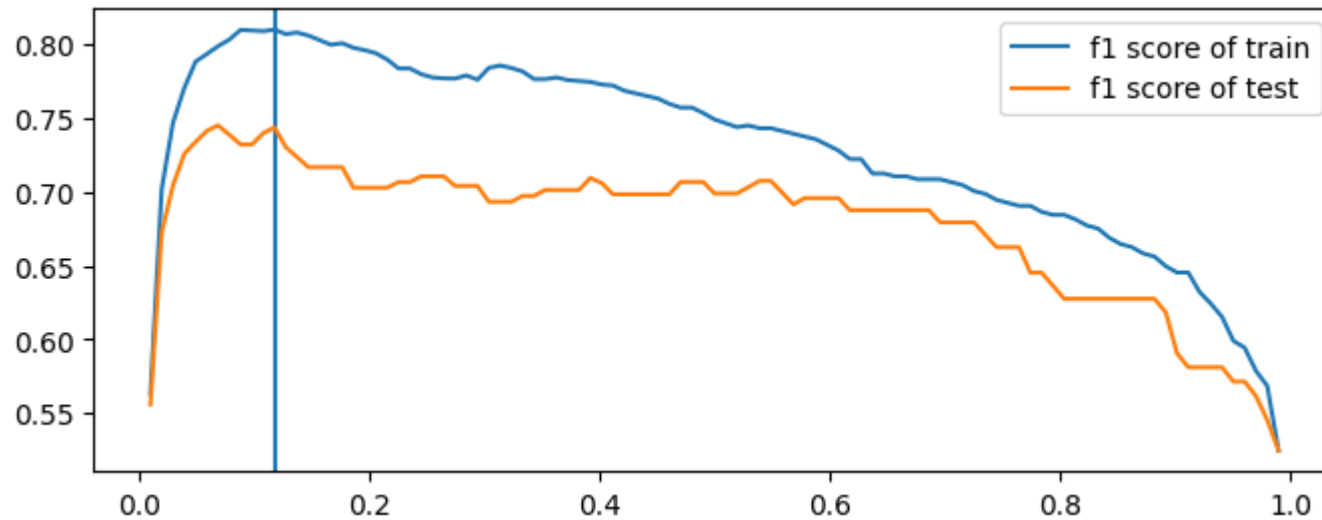
plt.figure(figsize=(8, 3))
plt.plot(thresholds, f1_scores,
         label='f1 score of train')
plt.axvline(thresholds[idx_max])
plt.plot(thresholds,
         [f1_score(df_test['Class'], prob_test >= threshold)

```

```

    for threshold in thresholds],
    label='f1 score of test')
plt.legend()
plt.show()

```



4. 데이터 가중치 차등 부여

학습시 클래스의 비중을 성능을 최대화 하는 방향으로 조정합니다.

※ sklearn에서는 클래스별 가중치를 조정하는 기능을 제공합니다.

[Ex.12]

학습 데이터 df_train에서 대상 변수 Class를 제외한

모든 변수를 입력 변수로 설정합니다.

df_train으로 로지스틱 회귀 모델을 클래스 가중치를

다음과 같이 바꾸어 가면서 학습합니다.

음의 클래스의 가중치를 1로 고정하고,

양의 클래스의 가중치를 1에서 10까지 0.5씩 증가시켜가면서

학습을 시키고 f1 score를 측정합니다.

가장 좋은 f1 score를 보인 경우를 확인해봅니다.

가중치별 학습데이터의 성능과 평가데이터의 성능 추이를 확인해봅니다.

In [35]:

```
'''
코드는 다양한 클래스 가중치 설정을 사용하여
로지스틱 회귀 모델을 학습시키고,
각각의 가중치에 대해 훈련 데이터와 테스트 데이터의
F1 점수를 계산합니다. 이를 통해 각 클래스 가중치가
모델 성능에 미치는 영향을 평가합니다.

### 1. 리스트 및 변수 초기화
- `f1_train_weight`, `f1_test_weight`:
  각각의 가중치 설정에 따른 훈련 데이터와
  테스트 데이터의 F1 점수를 저장할
  리스트를 초기화합니다.
- `X_cols`:
  `Class` 열을 제외한 훈련 데이터의
  입력 피쳐 이름 리스트를 생성합니다.
- `weights`:
  1에서 10.5까지 0.5 간격으로 증가하는 클래스 가중치
  배열을 생성합니다. 이는 클래스 1의 가중치를 나타내며,
  클래스 0의 가중치는 1로 고정됩니다.
- `f1_scores`: F1 점수를 저장할 리스트를 초기화합니다.

### 2. 클래스 가중치별 모델 학습 및 평가
- 반복문 (`for i in weights`):
  각 클래스 가중치에 대해 모델을 학습시키고 평가합니다.
  - 클래스 가중치 설정 (`class_weight={0: 1.0, 1: i}`):
    - 클래스 0의 가중치는 1.0으로 고정합니다.
    - 클래스 1의 가중치는 `i` 값으로 설정합니다.
  - `make_pipeline`:
    - `StandardScaler`: 데이터를 표준화합니다
```

```

        (평균을 0, 표준편차를 1로 변환).
    - `LogisticRegression(solver='lbfgs',
        class_weight={0: 1.0, 1: i})`:
        지정된 클래스 가중치를 사용하여
        로지스틱 회귀 모델을 생성합니다.
    - 모델 학습 (`clf_lr.fit`):
    - `df_train[X_cols]`: 입력 피쳐 데이터.
    - `df_train['Class']`: 레이블 데이터.
    - 훈련 데이터 예측 및 F1 점수 계산:
    - `prd = clf_lr.predict(df_train[X_cols].values)`:
        훈련 데이터에 대한 예측을 수행합니다.
    - `f1_train_weight.append(f1_score(
        df_train['Class'], prd))`:
        실제 레이블과 예측된 클래스를 비교하여
        F1 점수를 계산하고, 이를 리스트에 추가합니다.
    - 테스트 데이터 예측 및 F1 점수 계산:
    - `prd = clf_lr.predict(df_test[X_cols].values)`:
        테스트 데이터에 대한 예측을 수행합니다.
    - `f1_test_weight.append(f1_score(
        df_test['Class'], prd))`:
        실제 레이블과 예측된 클래스를 비교하여
        F1 점수를 계산하고, 이를 리스트에 추가합니다.

```

전체 코드 요약

1. 리스트 및 변수 초기화:
 - F1 점수 및 입력 피쳐 이름 리스트를 초기화합니다.
2. 클래스 가중치별 모델 학습 및 평가:
 - 다양한 클래스 가중치 설정에 대해
 - 로지스틱 회귀 모델을 학습시킵니다.
 - 각 설정에 대해 훈련 데이터와 테스트 데이터에
 - 대한 F1 점수를 계산하여 리스트에 추가합니다.

이를 통해 클래스 가중치가 모델 성능에 미치는 영향을 평가할 수 있으며, 최적의 가중치 설정을 찾을 수 있습니다. 이는 특히 클래스 불균형 문제를 다룰 때 유용합니다.

```

from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import f1_score, accuracy_score

```

```

f1_train_weight, f1_test_weight = list(), list()
X_cols = [i for i in df_train.columns if i != 'Class']
weights = np.arange(1, 10.5, 0.5)
f1_scores = list()

for i in weights:
    clf_lr = make_pipeline(
        StandardScaler(),
        LogisticRegression(solver='lbfgs',
                           class_weight={0: 1.0, 1: i})
    )
    clf_lr.fit(df_train[X_cols], df_train['Class'])
    prd = clf_lr.predict(df_train[X_cols].values)
    f1_train_weight.append(f1_score(df_train['Class'], prd))
    prd = clf_lr.predict(df_test[X_cols].values)
    f1_test_weight.append(f1_score(df_test['Class'], prd))

```

In [36]:

```

'''
코드는 다양한 클래스 가중치 설정에 따른
훈련 데이터와 테스트 데이터의 F1 점수를 시각화합니다.
최적의 가중치를 강조하고, 훈련 데이터와 테스트
데이터의 성능 차이를 비교할 수 있게 합니다.

### 1. 플롯 생성 및 크기 설정
- `plt.figure(figsize=(8, 3))`:
    가로 8인치, 세로 3인치의 새로운 그림(Figure)을 생성합니다.

### 2. 훈련 데이터의 F1 점수 플롯
- `plt.plot(weights, f1_train_weight,
             label='f1 score of train')`:
    - `weights`: x축 값, 다양한 클래스 가중치 배열.
    - `f1_train_weight`:
        y축 값, 각 클래스 가중치에 따른 훈련 데이터의 F1 점수.
    - `label='f1 score of train'`:
        훈련 데이터 F1 점수 플롯의 레이블을 설정합니다.

### 3. 최적의 훈련 데이터 F1 점수 가중치 강조
- `plt.axvline(weights[np.argmax(f1_train_weight)])`:
    - `weights[np.argmax(f1_train_weight)`:

```

```

        훈련 데이터에서 가장 높은 F1 점수를
        달성한 클래스 가중치.
- `np.argmax(f1_train_weight)` :
    `f1_train_weight` 리스트에서
    최댓값의 인덱스를 반환합니다.
- `plt.axvline` :
    최적의 클래스 가중치에서
    수직선을 그려 강조합니다.

### 4. 테스트 데이터의 F1 점수 플롯
- `plt.plot(weights, f1_test_weight, label='f1 score of test')` :
    - `weights` : x축 값, 다양한 클래스 가중치 배열.
    - `f1_test_weight` :
        y축 값, 각 클래스 가중치에 따른 테스트 데이터의 F1 점수.
    - `label='f1 score of test'` :
        테스트 데이터 F1 점수 플롯의 레이블을 설정합니다.

### 5. 범례 추가
- `plt.legend()` : 플롯에 범례를 추가하여 각 선이
    무엇을 나타내는지 설명합니다.

### 전체 코드 요약

1. 플롯 생성 및 크기 설정 :
    가로 8인치, 세로 3인치의 새로운 플롯을 생성합니다.
2. 훈련 데이터의 F1 점수 플롯 :
    다양한 클래스 가중치에 따른
    훈련 데이터의 F1 점수를 플롯합니다.
3. 최적의 훈련 데이터 F1 점수 가중치 강조 :
    훈련 데이터에서 가장 높은 F1 점수를 달성한
    클래스 가중치에서 수직선을 그려 강조합니다.
4. 테스트 데이터의 F1 점수 플롯 :
    다양한 클래스 가중치에 따른
    테스트 데이터의 F1 점수를 플롯합니다.
5. 범례 추가 :
    플롯에 범례를 추가하여 각 선의 의미를 설명합니다.

### 결과 분석

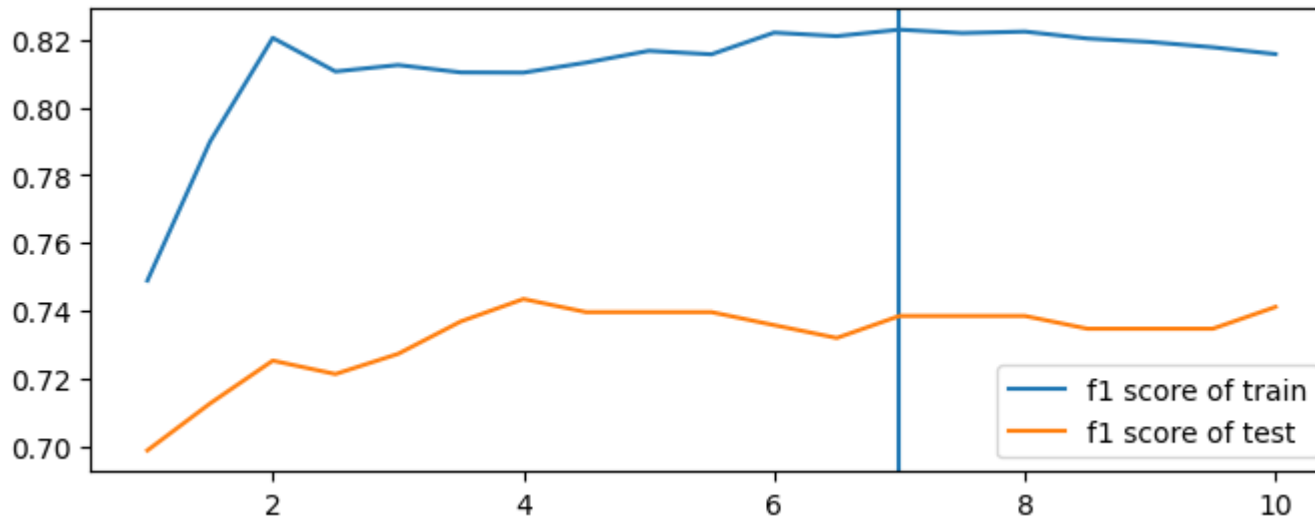
- 훈련 데이터 F1 점수 :
    다양한 클래스 가중치에 따른

```


- 훈련 데이터의 F1 점수를 보여줍니다.
- 테스트 데이터 F1 점수:
다양한 클래스 가중치에 따른
테스트 데이터의 F1 점수를 보여줍니다.
 - 최적의 클래스 가중치:
훈련 데이터에서 가장 높은 F1 점수를 달성한
클래스 가중치가 수직선으로 강조됩니다.

이 시각화를 통해 각 클래스 가중치 설정이
모델 성능에 미치는 영향을 쉽게
비교할 수 있습니다. 특히, 최적의
클래스 가중치를 선택하는 데 유용합니다.
'''

```
plt.figure(figsize=(8, 3))
plt.plot(weights, f1_train_weight, label='f1 score of train')
plt.axvline(weights[np.argmax(f1_train_weight)])
plt.plot(weights, f1_test_weight, label='f1 score of test')
plt.legend()
plt.show()
```



In [37]: '''

코드는 훈련 데이터에서 최적의 F1 점수를 달성한
가중치에 대응하는 테스트 데이터의 F1 점수를 반환합니다.

```

### 1. 최적의 훈련 데이터 F1 점수 인덱스 찾기
- `np.argmax(f1_train_weight)`:
  `f1_train_weight` 리스트에서
  최대값의 인덱스를 반환합니다.
  이는 훈련 데이터에서 최적의 F1 점수를
  달성한 가중치에 해당하는 인덱스입니다.

### 2. 테스트 데이터의 해당 F1 점수 반환
- `f1_test_weight[np.argmax(f1_train_weight)]:`
  훈련 데이터에서 최적의 F1 점수를 달성한
  가중치에 해당하는 인덱스를 사용하여
  테스트 데이터의 F1 점수를 찾습니다.
- `f1_test_weight`:
  테스트 데이터에서 각 클래스 가중치에 따른
  F1 점수를 저장한 리스트입니다.
- `np.argmax(f1_train_weight)`:
  최적의 F1 점수를 달성한 가중치에
  해당하는 인덱스를 찾습니다.

### 코드 요약
이 코드는 훈련 데이터에서 최적의 F1 점수를 달성한
가중치에 해당하는 인덱스를 사용하여
해당 가중치에서의 테스트 데이터의 F1 점수를 찾습니다.
이를 통해 최적의 훈련 데이터 F1 점수와
해당 가중치에서의 테스트 데이터 F1 점수를
비교할 수 있습니다. 이는 모델이 얼마나
일반화되었는지를 평가하는 데 도움이 됩니다.
'''

f1_test_weight[np.argmax(f1_train_weight)]

```

Out[37]: 0.7384615384615386

In []: